

Arquitectura RISC

Reduced Instruction Set Computer

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

Arquitectura RISC

- RISC-I (1981), RISC-II (1983), SOAR (1984), SPUR (1989), and RISC-V (2010)
- El conjunto de instrucciones del RISC-V fue desarrollado por la U. Berkeley
- Las instrucciones son Open & Free para usar en la Industria y Educación
- Actualmente la RISC-V Foundation (<http://riscv.org>) se encarga de su difusión
- Carece de micro-programa y las instrucciones se ejecutan en secuencia
- El compilador hace el mayor trabajo en la traducción de HLL a ASM

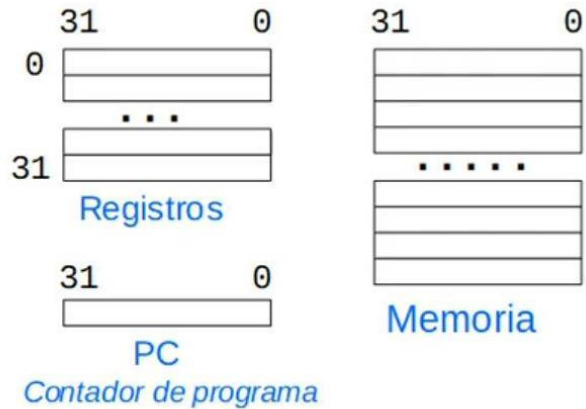
Arquitectura RISC-V

Versiones:

- 32 bits
- 64 bits
- 128 bits

RV32I

Palabras de 32 bits (*word*)



Arquitectura RISC-V

Versiones:

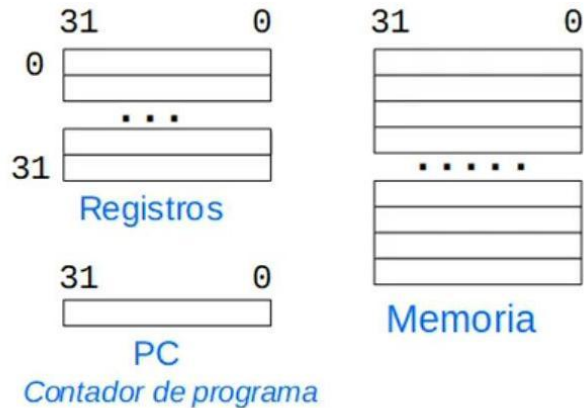
- 32 bits
- 64 bits
- 128 bits

Extensiones:

- I: Enteros (base)
- M: Multiplicación y división
- F, D: Punto flotante
- A: Atómicas
- C: Comprimidas
- V: Vector

RV32I

Palabras de 32 bits (*word*)



Arquitectura RISC-V

Versiones:

- 32 bits
- 64 bits
- 128 bits

Extensiones:

- I: Enteros (base)
- M: Multiplicación y división
- F, D: Punto flotante
- A: Atómicas
- C: Comprimidas
- V: Vector



RV32I: RISC-V 32 bits entero

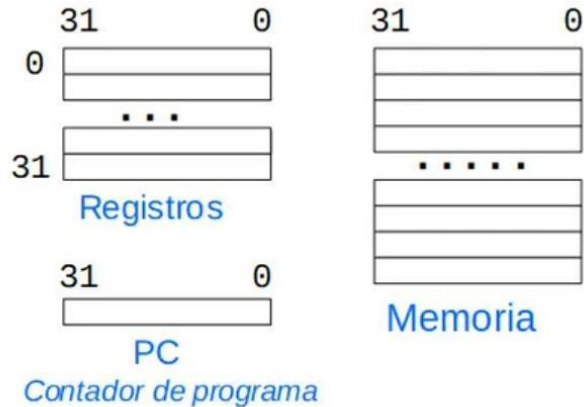
RV32IM: RISC-V 32 bits entero y multiplicación

RV64IMAFD

.....

RV32I

Palabras de 32 bits (*word*)



Arquitectura RISC-V

Versiones:

- 32 bits
- 64 bits
- 128 bits

Extensiones:

- I: Enteros (base)
- M: Multiplicación y división
- F, D: Punto flotante
- A: Atómicas
- C: Comprimidas
- V: Vector



RV32I: RISC-V 32 bits entero

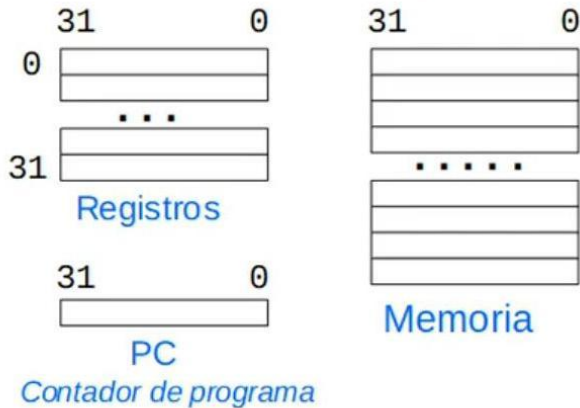
RV32IM: RISC-V 32 bits entero y multiplicación

RV64IMAFD

.....

RV32I

Palabras de 32 bits (*word*)



Arquitectura RISC-V

Versiones:

- 32 bits
- 64 bits
- 128 bits

Extensiones:

- I: Enteros (base)
- M: Multiplicación y división
- F, D: Punto flotante
- A: Atómicas
- C: Comprimidas
- V: Vector



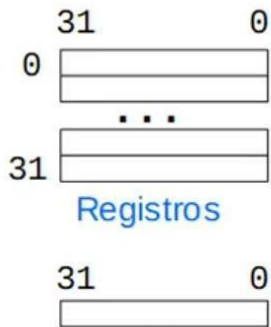
RV32I: RISC-V 32 bits entero

RV32IM: RISC-V 32 bits entero y multiplicación
RV64IMAFD

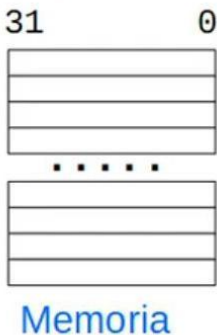
.....

RV32I

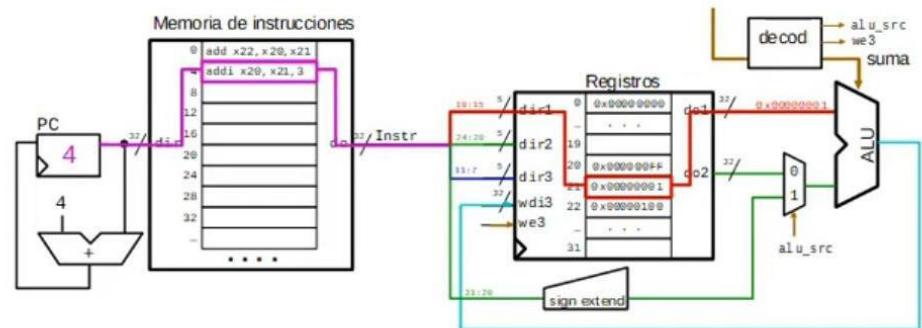
Palabras de 32 bits (*word*)



Contador de programa

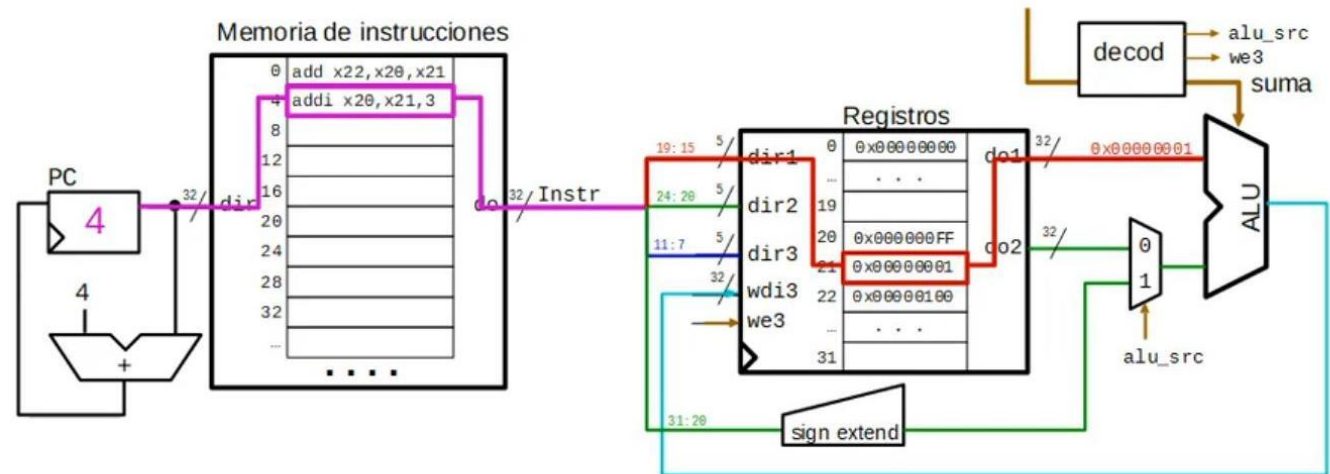


Diseño básico de la microarquitectura



Arquitectura Básica RISC-V

Aplicaciones
Sistema operativo
Conjunto de instrucciones
Microarquitectura
RTL
Puertas lógicas
Circuitos analógicos
Componentes
Física



Registros

#	Name	Usage
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporaries (Caller-save registers)
x6	t1	
x7	t2	
x8	s0/fp	Saved register / Frame pointer
x9	s1	Saved register
x10	a0	Function arguments / Return values
x11	a1	
x12	a2	Function arguments
x13	a3	
x14	a4	
x15	a5	

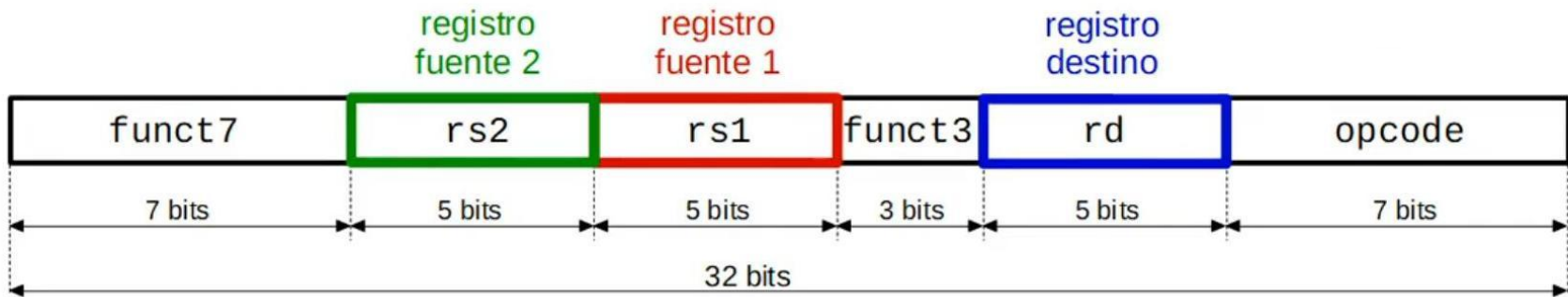
#	Name	Usage
x16	a6	Function arguments
x17	a7	
x18	s2	Saved registers (Callee-save registers)
x19	s3	
x20	s4	
x21	s5	
x22	s6	
x23	s7	
x24	s8	
x25	s9	
x26	s10	
x27	s11	
x28	t3	Temporaries (Caller-save registers)
x29	t4	
x30	t5	
x31	t6	
	pc	Program counter

Instrucciones con Registros

	Tipo R	
<i>Add</i>	add	Aritméticas
<i>Subtract</i>	sub	
<i>AND</i>	and	A nivel de bits
<i>OR</i>	or	
<i>Exclusive OR</i>	xor	
<i>Set if Less Than</i>	slt	Comparaciones
<i>Set if Less Than Unsigned</i>	sltu	
<i>Shift Left Logic</i>	sll	Desplazamientos
<i>Shift Right Logic</i>	srl	
<i>Shift Right Arithmetic</i>	sra	

Instrucciones con Registros

add x19, x20, x21 // $x19 \leftarrow x20 + x21$

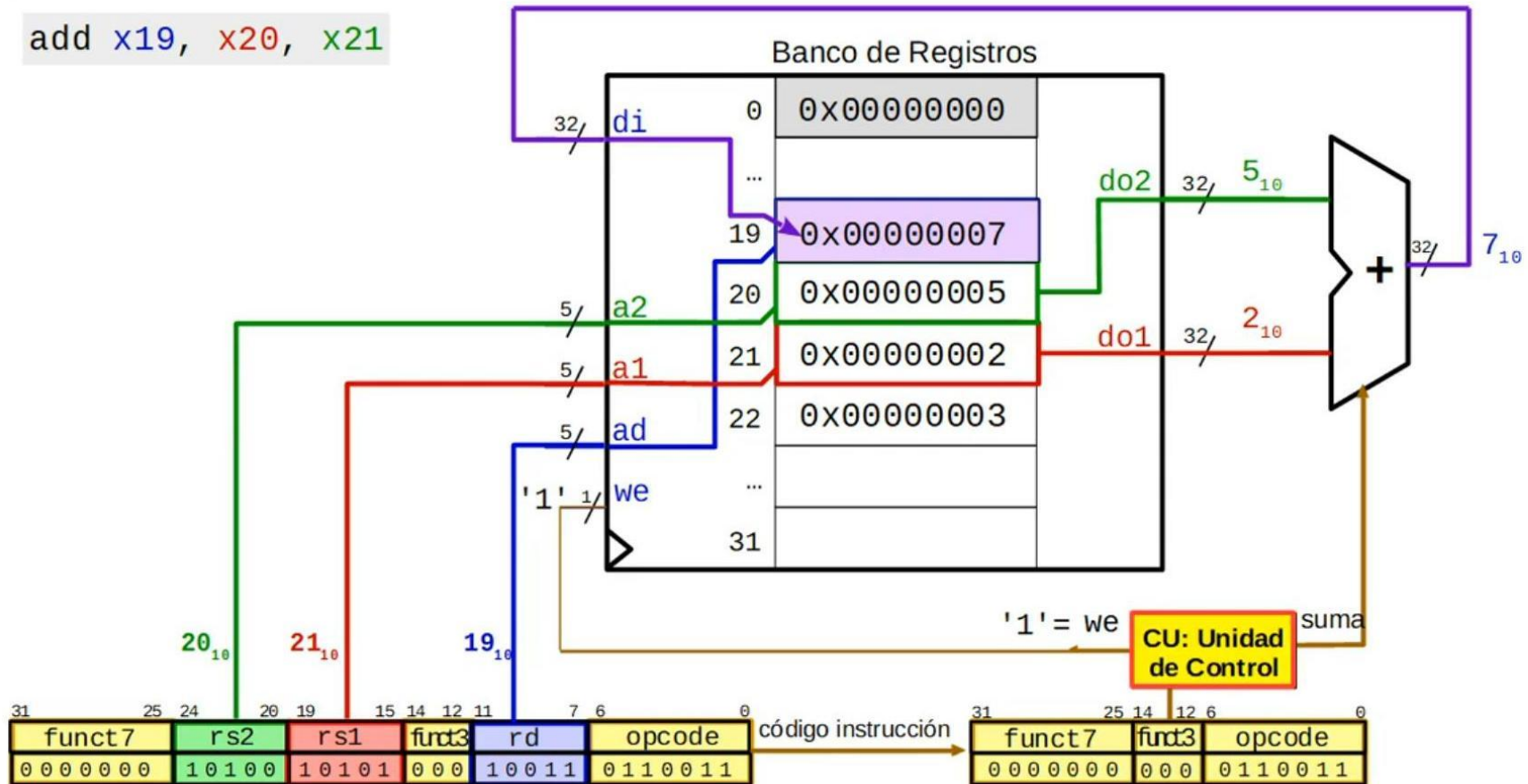


Código de operación:

funct7 funct3 opcode

Instrucciones con Registros

add x19, x20, x21



Instrucciones con Registros

Tipo R	funct7	rs2	rs1	funct3	rd	opcode
add	0 0 0 0 0 0 0	rs2	rs1	0 0 0	rd	0 1 1 0 0 1 1
sub	0 1 0 0 0 0 0	rs2	rs1	0 0 0	rd	0 1 1 0 0 1 1
and	0 0 0 0 0 0 0	rs2	rs1	1 1 1	rd	0 1 1 0 0 1 1
or	0 0 0 0 0 0 0	rs2	rs1	1 1 0	rd	0 1 1 0 0 1 1
xor	0 0 0 0 0 0 0	rs2	rs1	1 0 0	rd	0 1 1 0 0 1 1
slt	0 0 0 0 0 0 0	rs2	rs1	0 1 0	rd	0 1 1 0 0 1 1
sltu	0 0 0 0 0 0 0	rs2	rs1	0 1 1	rd	0 1 1 0 0 1 1
sll	0 0 0 0 0 0 0	rs2	rs1	0 0 1	rd	0 1 1 0 0 1 1
srl	0 0 0 0 0 0 0	rs2	rs1	1 0 1	rd	0 1 1 0 0 1 1
sra	0 1 0 0 0 0 0	rs2	rs1	1 0 1	rd	0 1 1 0 0 1 1

Instrucciones con Registros

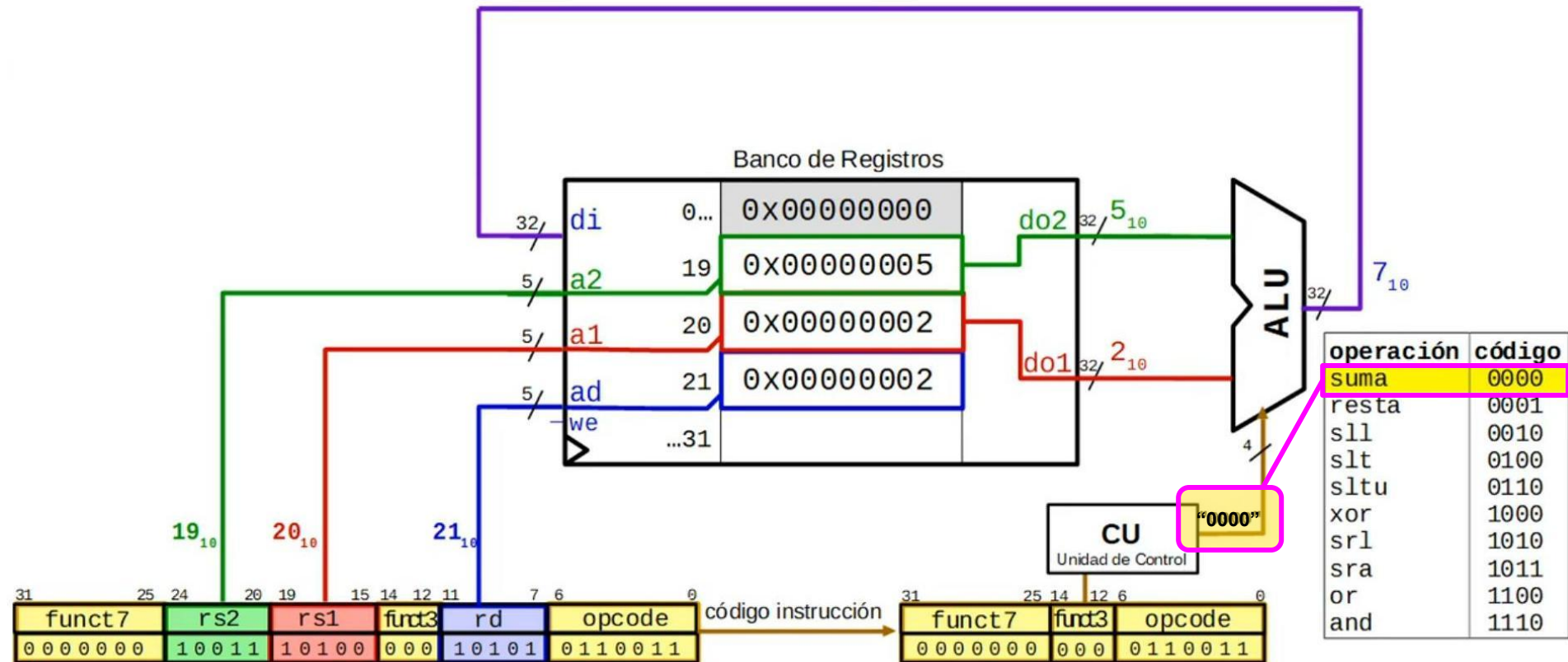
Tipo R	func7							rs2	rs1	func3			rd	opcode						
add	0	0	0	0	0	0	0	rs2	rs1	0	0	0	rd	0	1	1	0	0	1	1
sub	0	1	0	0	0	0	0	rs2	rs1	0	0	0	rd	0	1	1	0	0	1	1
and	0	0	0	0	0	0	0	rs2	rs1	1	1	1	rd	0	1	1	0	0	1	1
or	0	0	0	0	0	0	0	rs2	rs1	1	1	0	rd	0	1	1	0	0	1	1
xor	0	0	0	0	0	0	0	rs2	rs1	1	0	0	rd	0	1	1	0	0	1	1
slt	0	0	0	0	0	0	0	rs2	rs1	0	1	0	rd	0	1	1	0	0	1	1
sltu	0	0	0	0	0	0	0	rs2	rs1	0	1	1	rd	0	1	1	0	0	1	1
sll	0	0	0	0	0	0	0	rs2	rs1	0	0	1	rd	0	1	1	0	0	1	1
srl	0	0	0	0	0	0	0	rs2	rs1	1	0	1	rd	0	1	1	0	0	1	1
sra	0	1	0	0	0	0	0	rs2	rs1	1	0	1	rd	0	1	1	0	0	1	1

Instrucciones con Registros

Tipo R	funct7							rs2			rs1			funct3			rd		opcode																									
add	0	0	0	0	0	0	0	<table><tr><th>operación</th><th>código</th></tr><tr><td>suma</td><td>0000</td></tr><tr><td>resta</td><td>0001</td></tr><tr><td>sll</td><td>0010</td></tr><tr><td>slt</td><td>0100</td></tr><tr><td>sltu</td><td>0110</td></tr><tr><td>xor</td><td>1000</td></tr><tr><td>srl</td><td>1010</td></tr><tr><td>sra</td><td>1011</td></tr><tr><td>or</td><td>1100</td></tr><tr><td>and</td><td>1110</td></tr></table>			operación	código	suma	0000	resta	0001	sll	0010	slt	0100	sltu	0110	xor	1000	srl	1010	sra	1011	or	1100	and	1110	0 0 0			rd		0 1 1 0 0 1 1						
operación	código																																											
suma	0000																																											
resta	0001																																											
sll	0010																																											
slt	0100																																											
sltu	0110																																											
xor	1000																																											
srl	1010																																											
sra	1011																																											
or	1100																																											
and	1110																																											
sub	0	1	0	0	0	0	0	0 0 0			rd		0 1 1 0 0 1 1																															
and	0	0	0	0	0	0	0	1 1 1			rd		0 1 1 0 0 1 1																															
or	0	0	0	0	0	0	0	1 1 0			rd		0 1 1 0 0 1 1																															
xor	0	0	0	0	0	0	0	1 0 0			rd		0 1 1 0 0 1 1																															
slt	0	0	0	0	0	0	0	0 1 0			rd		0 1 1 0 0 1 1																															
sltu	0	0	0	0	0	0	0	0 1 1			rd		0 1 1 0 0 1 1																															
sll	0	0	0	0	0	0	0	0 1 0			rd		0 1 1 0 0 1 1																															
srl	0	0	0	0	0	0	0	0 1 1			rd		0 1 1 0 0 1 1																															
sra	0	1	0	0	0	0	0	0 0 1			rd		0 1 1 0 0 1 1																															
	0	0	0	0	0	0	0	1 0 1			rd		0 1 1 0 0 1 1																															
	0	1	0	0	0	0	0	1 0 1			rd		0 1 1 0 0 1 1																															

Instrucciones con Registros

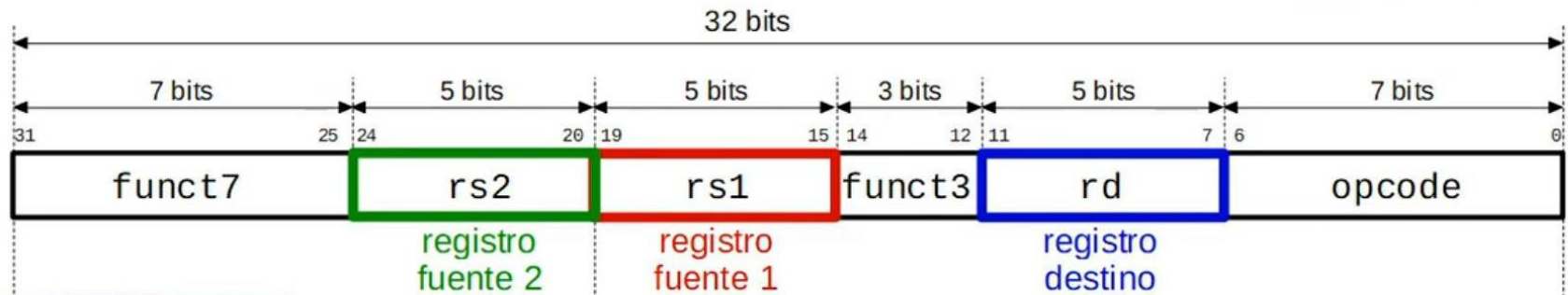
```
add x21, x20, x19
```



Instrucciones con Registros vs Constantes

add rd, rs1, rs2

Formato instrucción tipo R



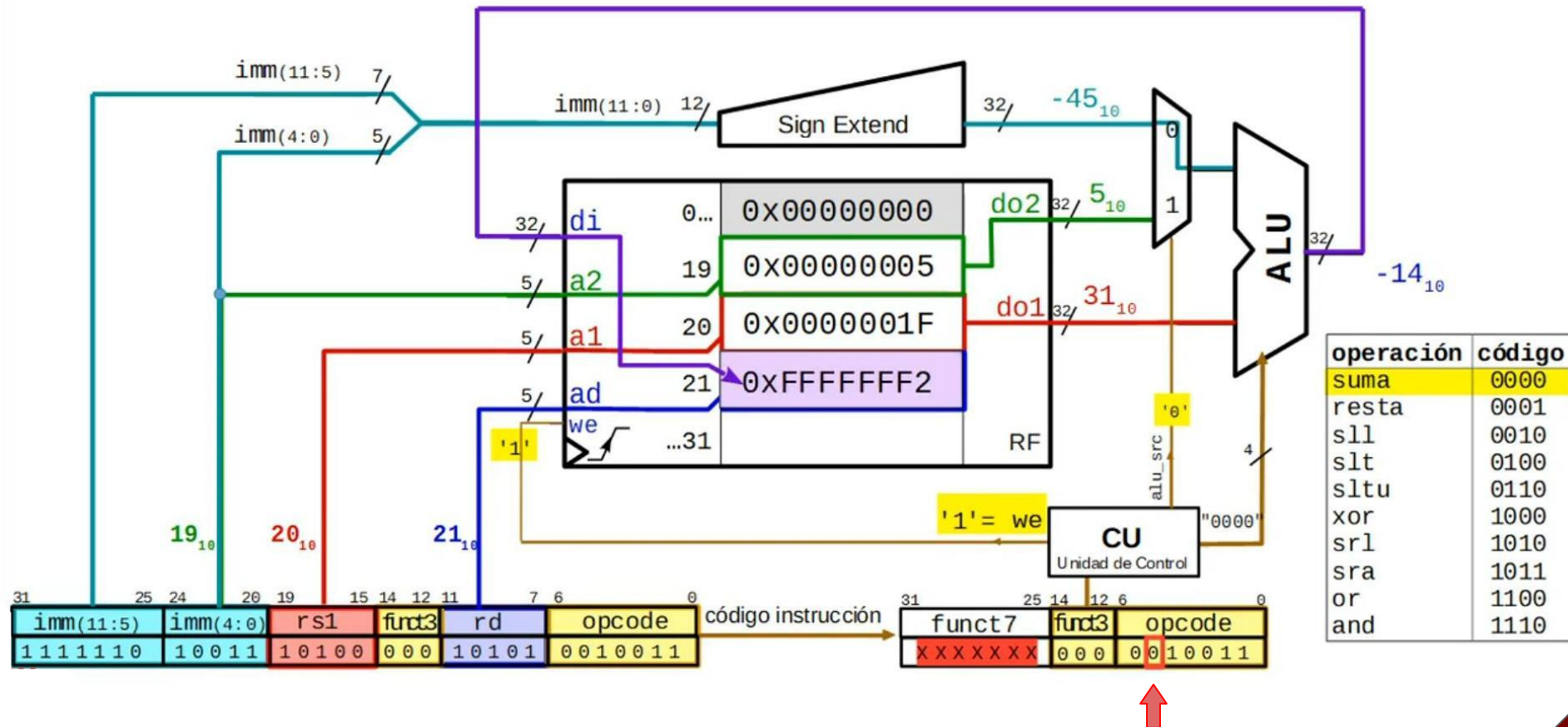
addi rd, rs1, imm

Formato instrucción tipo I



Instrucciones con Constantes

```
addi x21, x20, -45
```



Instrucciones con Constantes

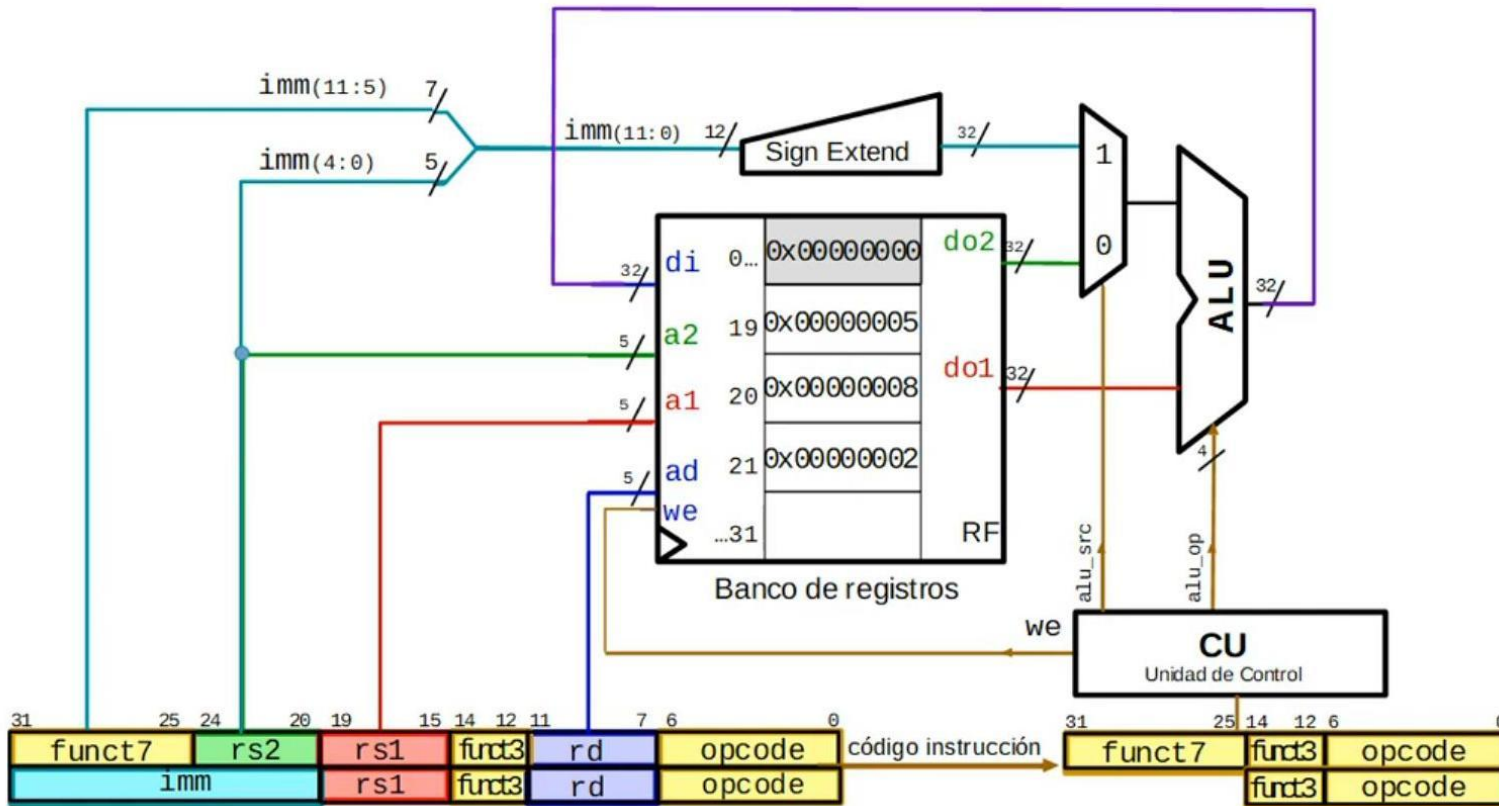
Tipo I

addi	imm(11:0)								rs1	0 0 0	rd	0 0 1 0 0 1 1
andi	imm(11:0)								rs1	1 1 1	rd	0 0 1 0 0 1 1
ori	imm(11:0)								rs1	1 1 0	rd	0 0 1 0 0 1 1
xori	imm(11:0)								rs1	1 0 0	rd	0 0 1 0 0 1 1
slti	imm(11:0)								rs1	0 1 0	rd	0 0 1 0 0 1 1
sltui	imm(11:0)								rs1	0 1 1	rd	0 0 1 0 0 1 1
slli	0 0 0 0 0 0 0 0	shamt				rs1	0 0 1	rd	0 0 1 0 0 1 1			
srli	0 0 0 0 0 0 0 0	shamt				rs1	1 0 1	rd	0 0 1 0 0 1 1			
srai	0 1 0 0 0 0 0 0	shamt				rs1	1 0 1	rd	0 0 1 0 0 1 1			
31 25 24 20 19 15 14 12 11 7 6 0												

Shift Amount → 5 bits desplazan los 32 bits del registro

diferencia con instrucciones tipo R

Instrucciones con Registros o Constantes



operación	código
suma	0000
resta	0001
sll	0010
slt	0100
sltu	0110
xor	1000
srl	1010
sra	1011
or	1100
and	1110

Instrucciones con Registros y Constantes

	Tipo R	Tipo I
<i>Add</i>	add	addi
<i>Subtract</i>	sub	—
<i>AND</i>	and	andi
<i>OR</i>	or	ori
<i>Exclusive OR</i>	xor	xori
<i>Set if Less Than</i>	slt	slti
<i>Set if Less Than Unsigned</i>	sltu	sltui
<i>Shift Left Logic</i>	sll	slli
<i>Shift Right Logic</i>	srl	srli
<i>Shift Right Arithmetic</i>	sra	srai

Aritméticas

A nivel de bits

Comparaciones

Desplazamientos

Instrucciones de Carga y Almacenamiento

Tipo I: Carga de memoria a registro

lw	<i>Load Word</i>
lhu	<i>Load Halfword, Unsigned</i>
lbu	<i>Load Byte, Unsigned</i>
lh	<i>Load Halfword</i>
lb	<i>Load Byte</i>

Tipo S: Almacenamiento, de registro a memoria

sw	<i>Store Word</i>
sh	<i>Store Halfword</i>
sb	<i>Store Byte</i>

Instrucciones de Carga

	31	25	24	20	19	15	14	12	11	7	6	0
Tipo I	imm				rs1	funct3			rd	opcode		
lb	imm(11:0)				rs1	0 0 0			rd	0 0 0 0 0 1 1		
lh	imm(11:0)				rs1	0 0 1			rd	0 0 0 0 0 1 1		
lw	imm(11:0)				rs1	0 1 0			rd	0 0 0 0 0 1 1		
lbu	imm(11:0)				rs1	1 0 0			rd	0 0 0 0 0 1 1		
lhu	imm(11:0)				rs1	1 0 1			rd	0 0 0 0 0 1 1		

Instrucciones de Carga

Cargar media palabra **sin signo** de memoria a registro

`lhu rd, offset(rs1)`

Load Halfword, Unsigned

No son 8 medias palabras más
2 bytes = 1 medias palabras
8 bytes → 4 medias palabras

Ejemplo `lhu x22, 8(x19)`

Registros

...	
x19	24
x20	
x21	
x22	0000 0FA1
x23	
...	



24+8=32

Memoria

0				
4				
8				
12				
16				
20				
24				
28				
32				0FA1
36				
40				
44				
48				
52				
...				

Instrucciones de Carga

Cargar media palabra **sin signo** de memoria a registro

`lhu rd, offset(rs1)`

Load Halfword, Unsigned

No son 8 medias palabras más
2 bytes = 1 medias palabras
8 bytes → 4 medias palabras

Ejemplo `lhu x22, 8(x19)`

Registros

...	
x19	24
x20	
x21	
x22	0000 0FA1
x23	
...	



24+8=32

Memoria

0				
4				
8				
12				
16				
20				
24				
28				
32				0FA1
36				
40				
44				
48				
52				
...				

Instrucciones de Carga

Cargar media palabra **sin signo** de memoria a registro

`lhu rd, offset(rs1)`

Load Halfword, Unsigned → Son 5 bytes más

Ejemplo `lhu x22, 5(x19)`

... Registros

x19	28
x20	
x21	
x22	0000 FFA1
x23	
...	



28+5=33

Memoria

0				
4				
8				
12				
16				
20				
24				
28				
32			FFA1	33
36				
40				
44				
48				
52				
...				

Instrucciones de Carga

Cargar byte **sin signo** de memoria a registro

```
lbu rd, offset(rs1)
```

Load Byte, Unsigned

Ejemplo lbu **x22**, **5** (**x19**)

... Registros

x19	28
x20	
x21	
x22	0000 00F2
x23	
...	



28+5=33

Memoria

0				
4				
8				
12				
16				
20				
24				
28				
32		F2		33
36				
40				
44				
48				
52				
...				

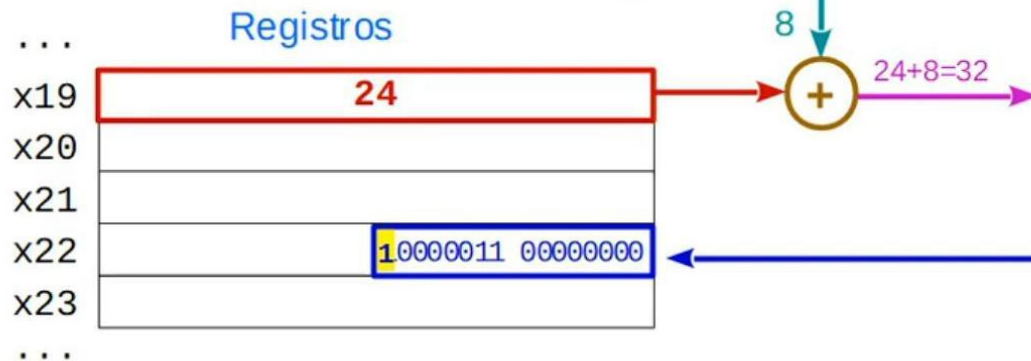
Instrucciones de Carga

Cargar media palabra **con signo** de memoria a registro

```
lh rd, offset(rs1)
```

Load Halfword

Ejemplo `lh x22, 8(x19)`



Memoria

0				
4				
8				
12				
16				
20				
24				
28				
32			10000011	00000000
36				
40				
44				
48				
52				
...				

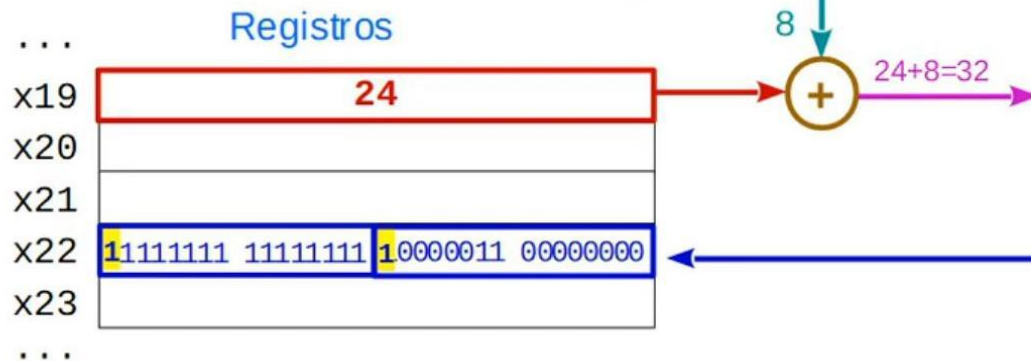
Instrucciones de Carga

Cargar media palabra **con signo** de memoria a registro

```
lh rd, offset(rs1)
```

Load Halfword

Ejemplo `lh x22, 8(x19)`



Memoria

0				
4				
8				
12				
16				
20				
24				
28				
32		10000011	00000000	
36				
40				
44				
48				
52				
...				

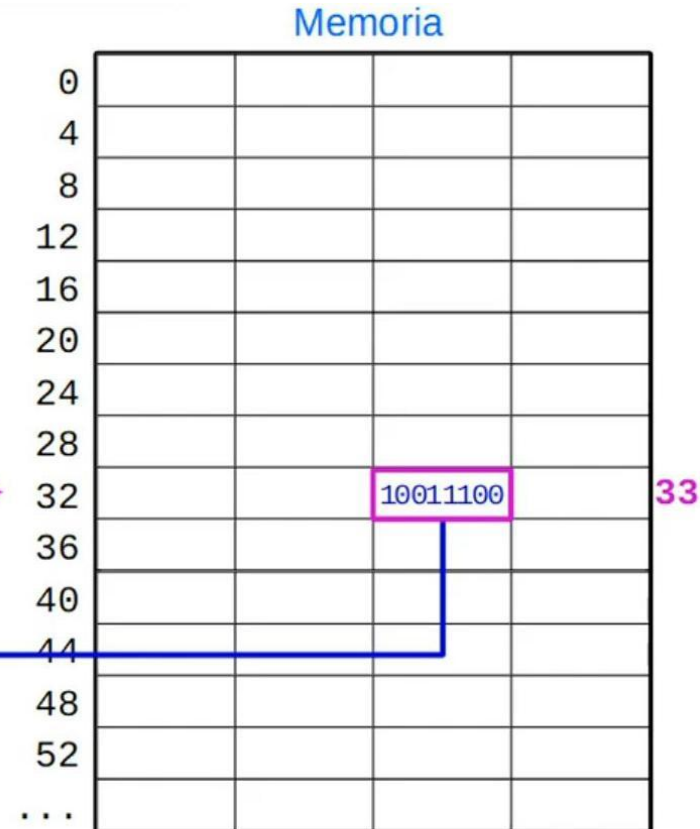
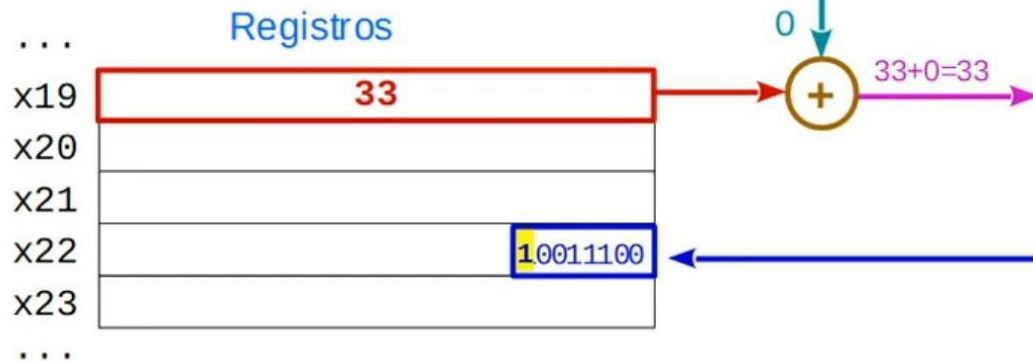
Instrucciones de Carga

Cargar byte **con signo** de memoria a registro

```
lb rd, offset(rs1)
```

Load Byte

Ejemplo `lb x22, 0(x19)`



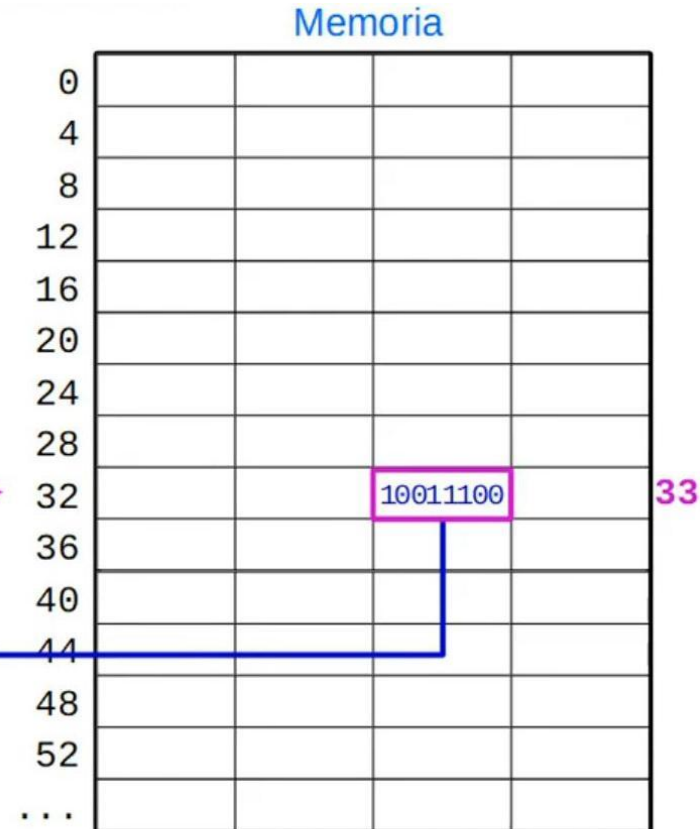
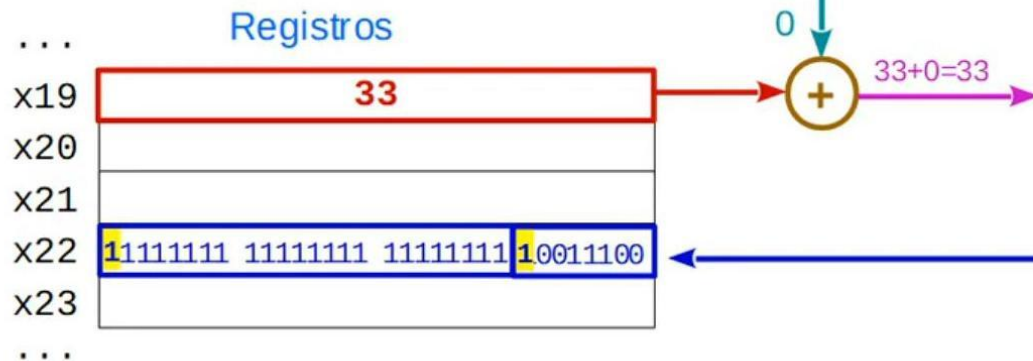
Instrucciones de Carga

Cargar byte **con signo** de memoria a registro

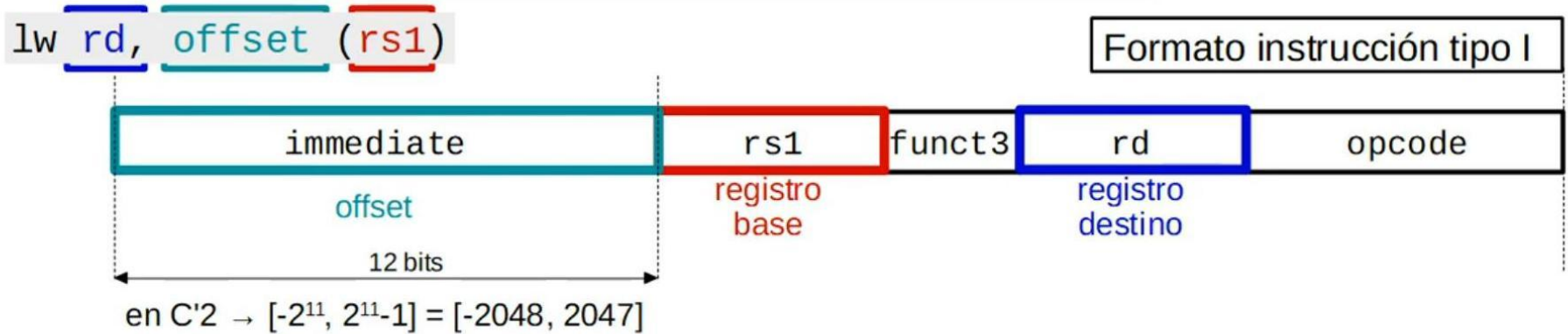
```
lb rd, offset(rs1)
```

Load Byte

Ejemplo `lb x22, 0(x19)`



Instrucciones de Carga



rs1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

64₁₀

imm

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	

-32₁₀

dir

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	

32₁₀

rd $\leftarrow$ Memo[dir]

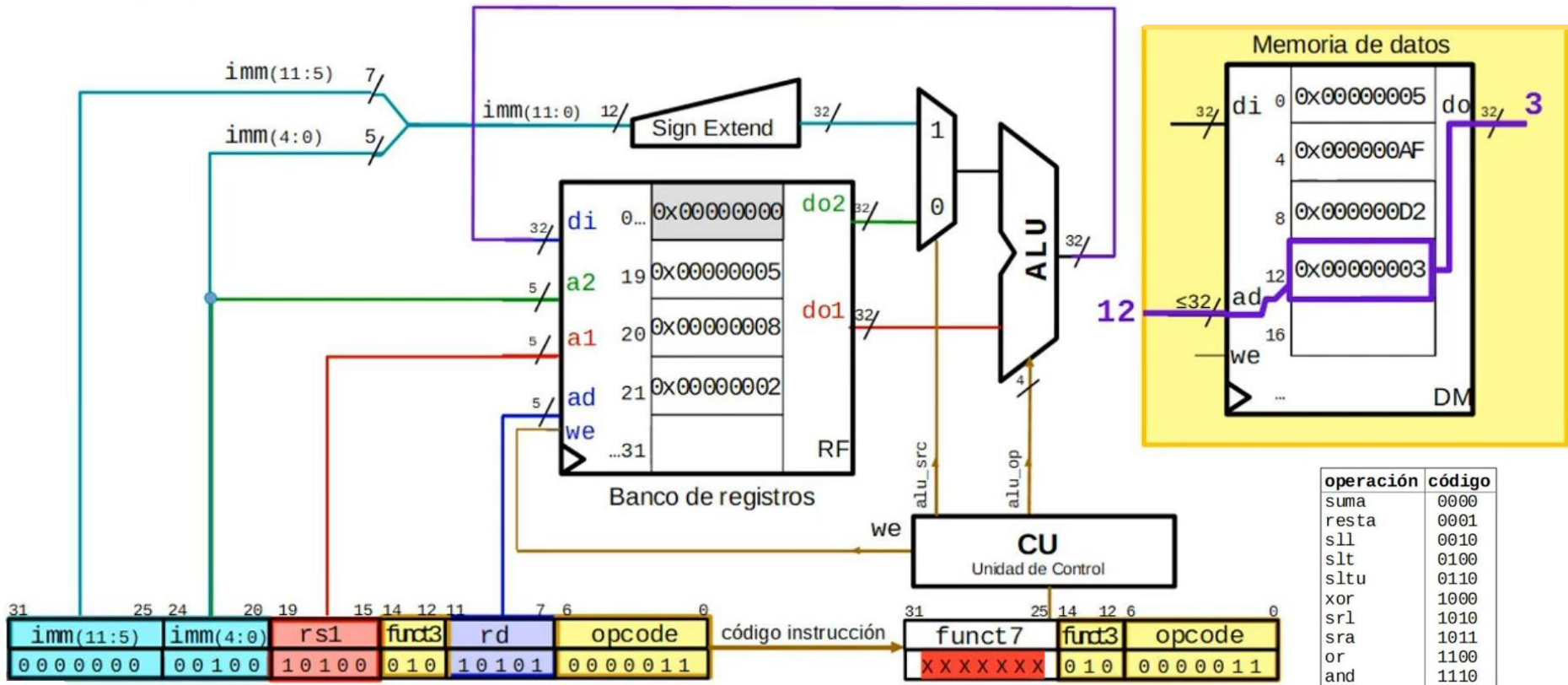
rd $\leftarrow$ Memo[32]

rd

Memo[32]																															
----------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

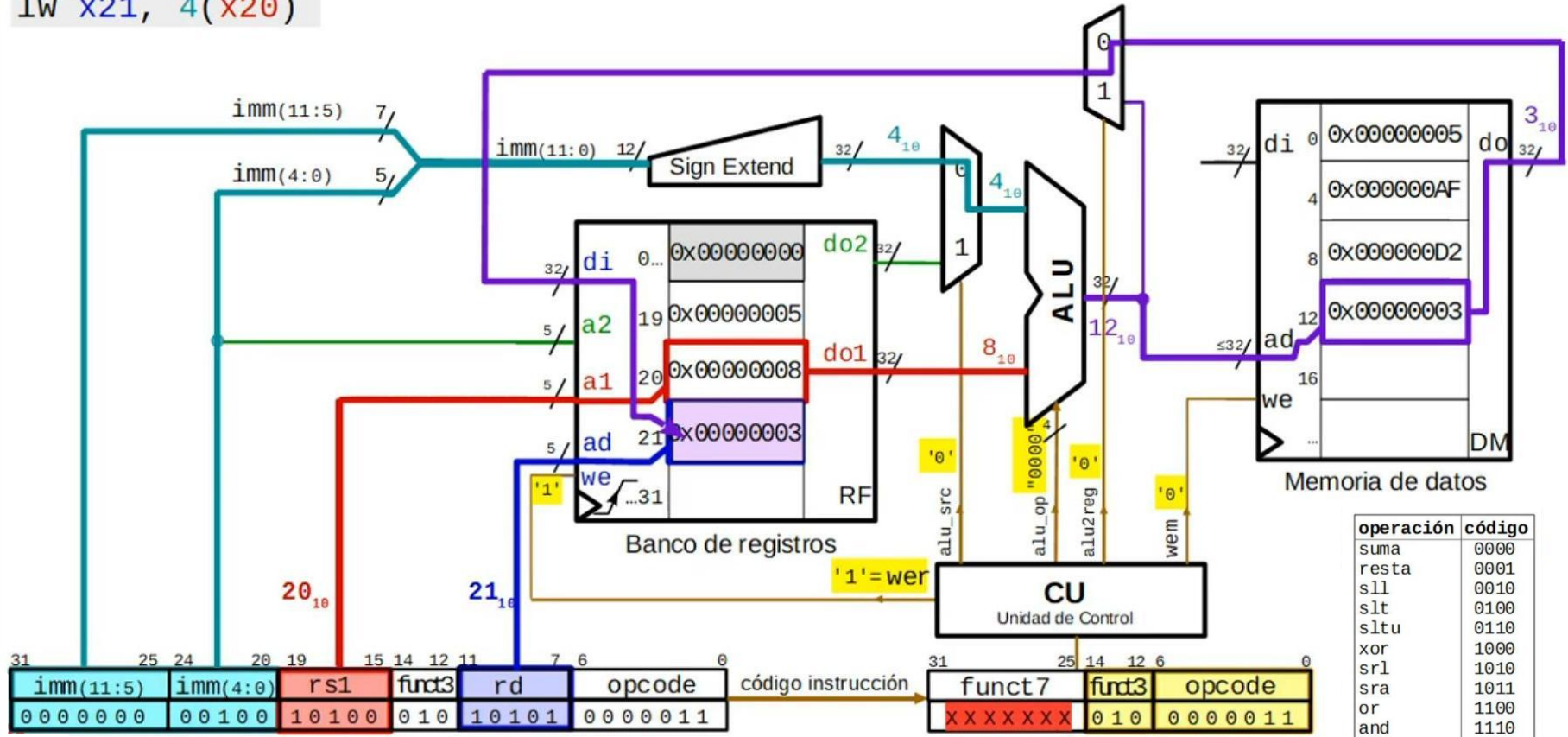
Instrucciones de Carga

```
lw x21, 4(x20)
```

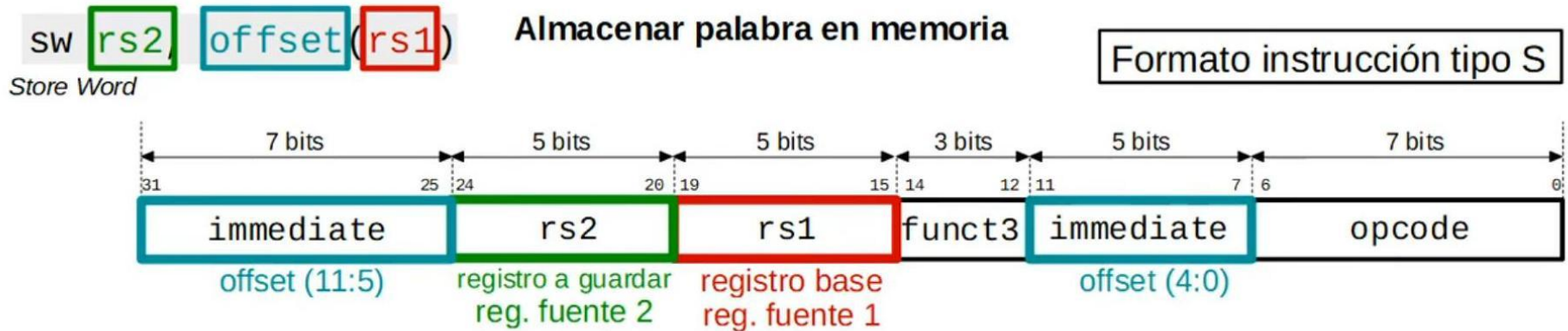
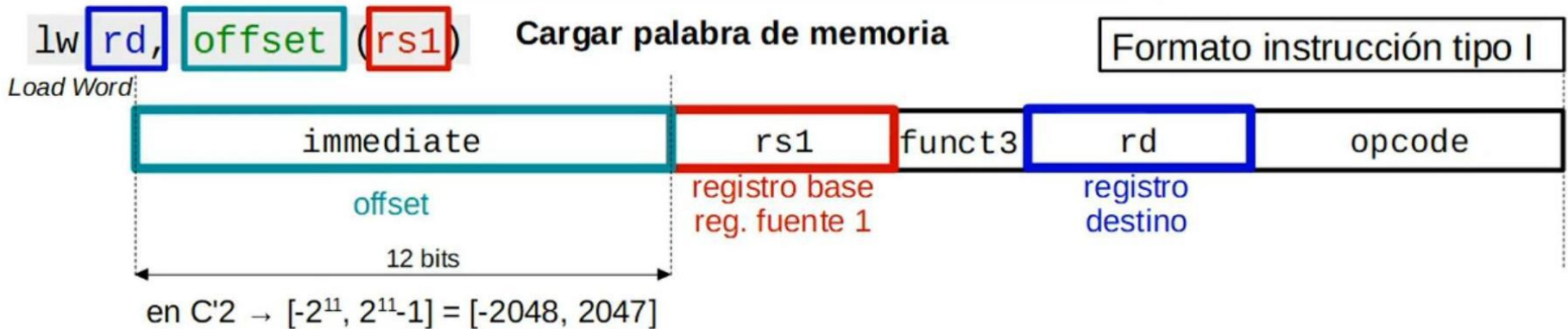


Instrucciones de Carga

```
lw x21, 4(x20)
```



Instrucciones de Carga y Almacenamiento



Comparativa de Formatos de Instrucción

Dos registros fuente

Sin registro destino



31		25 24		20 19		15 14		12 11		7 6		0	
imm (11:5)		rs2		rs1		funct3		imm (4:0)		opcode			
imm (11:5)		rs2		rs1		0 0 0		imm (4:0)		0 1 0 0 0 1 1			
imm (11:5)		rs2		rs1		0 0 1		imm (4:0)		0 1 0 0 0 1 1			
imm (11:5)		rs2		rs1		0 1 0		imm (4:0)		0 1 0 0 0 1 1			

	31	25	24	20	19	15	14	12	11	7	6				
	imm (11:5)					imm (4:0)		rs1		funct3		rd		opcode	
	funct7					rs2		rs1		funct3		rd		opcode	

Instrucciones de Almacenamiento

	31	25	24	20	19	15	14	12	11	7	6	0
Tipo S	imm (11:5)		rs2		rs1		funct3		imm (4:0)		opcode	
sb	imm (11:5)		rs2		rs1		0 0 0		imm (4:0)		0 1 0 0 0 1 1	
sh	imm (11:5)		rs2		rs1		0 0 1		imm (4:0)		0 1 0 0 0 1 1	
sw	imm (11:5)		rs2		rs1		0 1 0		imm (4:0)		0 1 0 0 0 1 1	

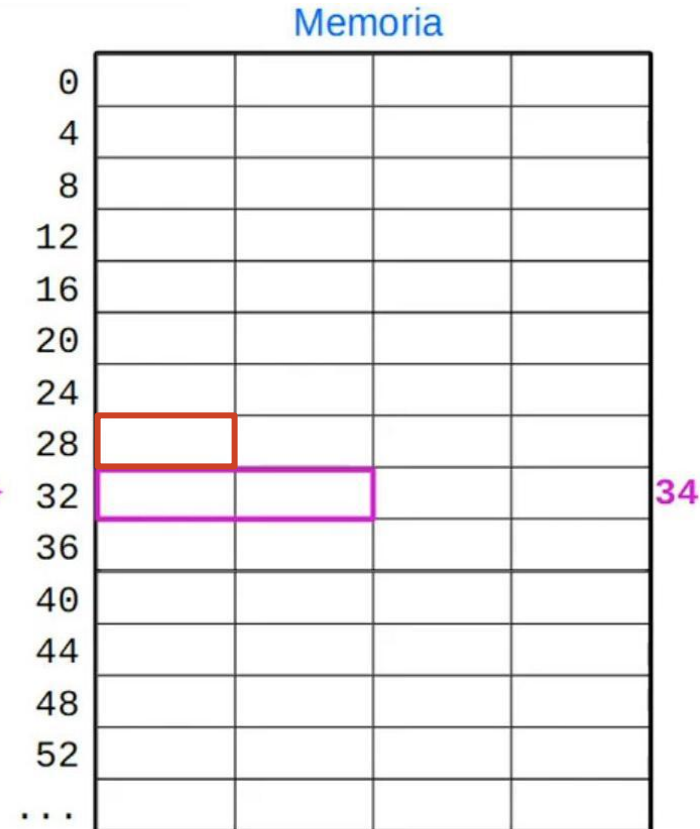
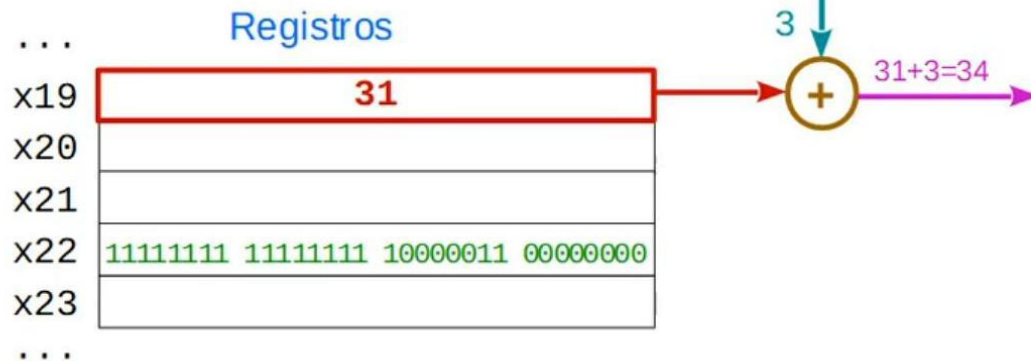
Instrucciones de Almacenamiento

Almacenar media palabra de registro a memoria

```
sh rs2, offset(rs1)
```

Store Halfword

Ejemplo sh x22, 3 (x19)



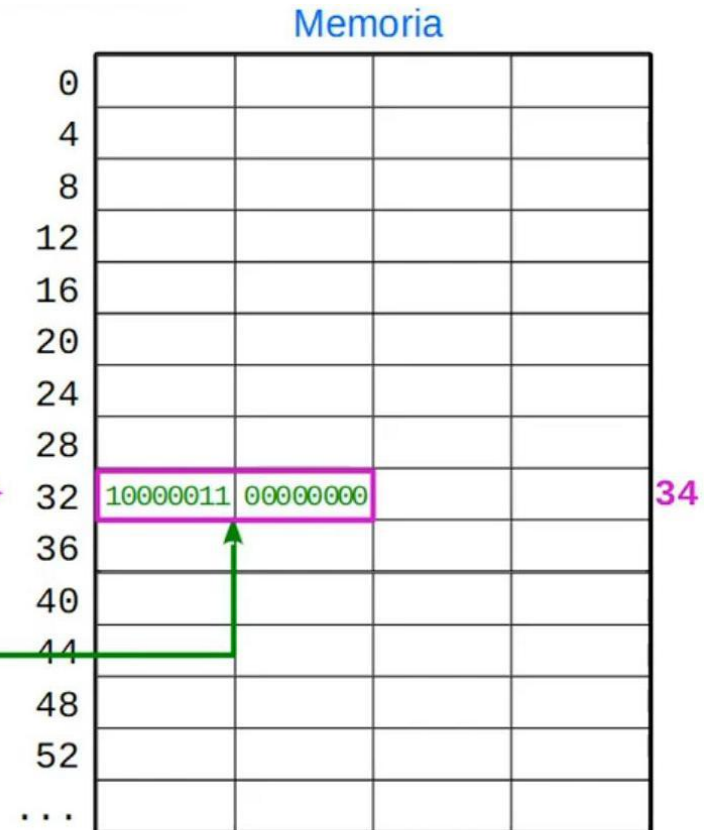
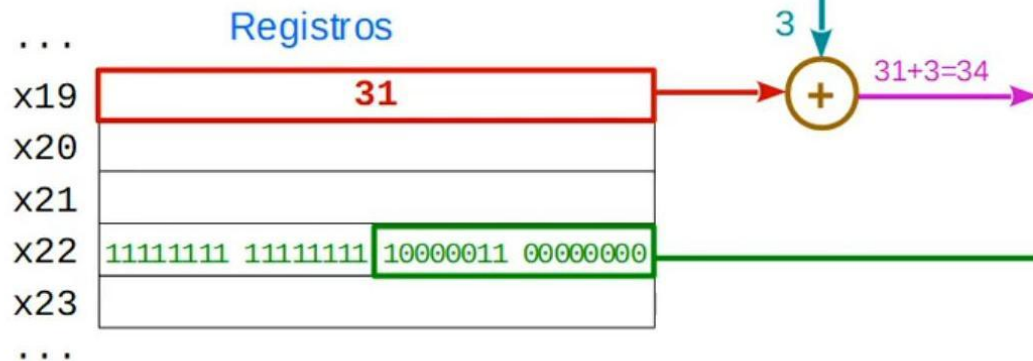
Instrucciones de Almacenamiento

Almacenar media palabra de registro a memoria

```
sh rs2, offset(rs1)
```

Store Halfword

Ejemplo sh x22, 3 (x19)



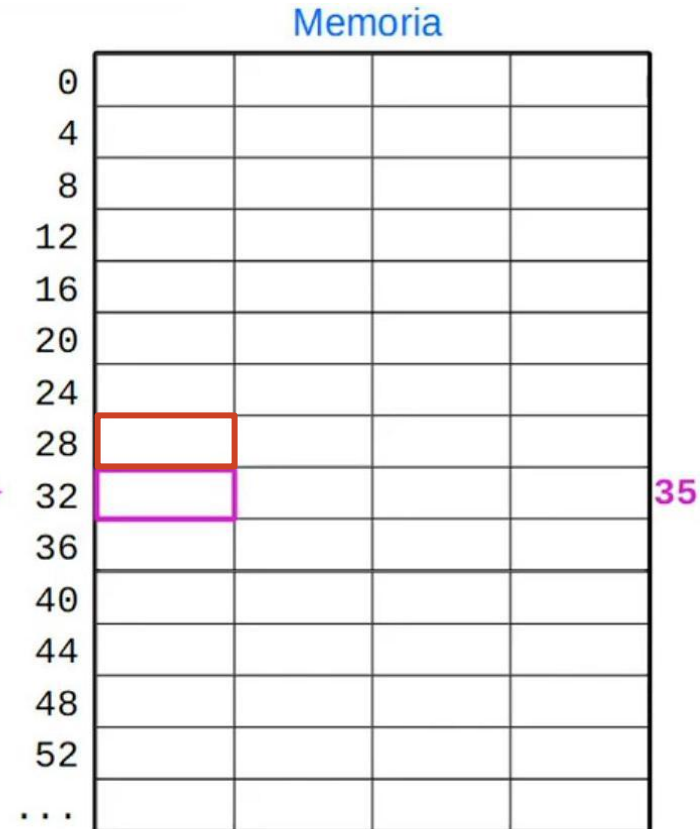
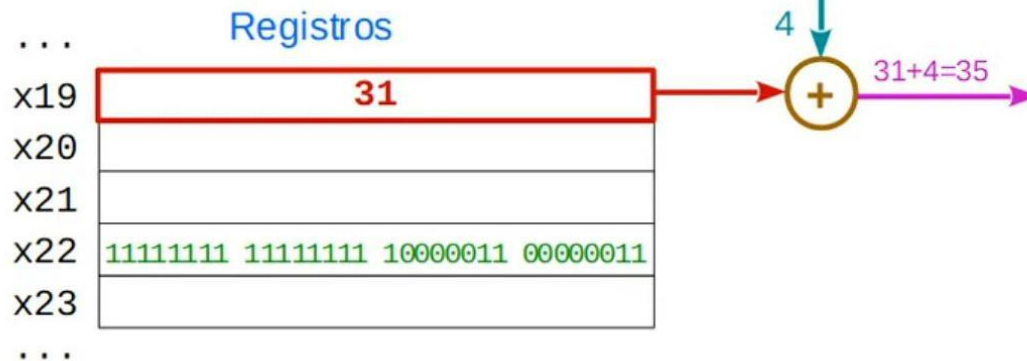
Instrucciones de Almacenamiento

Almacenar byte de registro a memoria

```
sb rs2, offset(rs1)
```

Store Byte

Ejemplo sb x22, 4(x19)



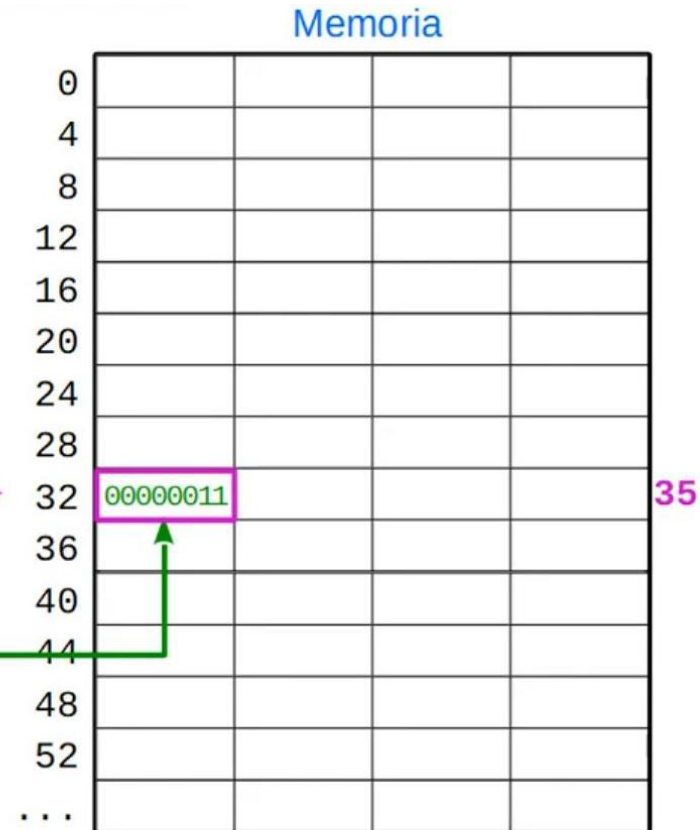
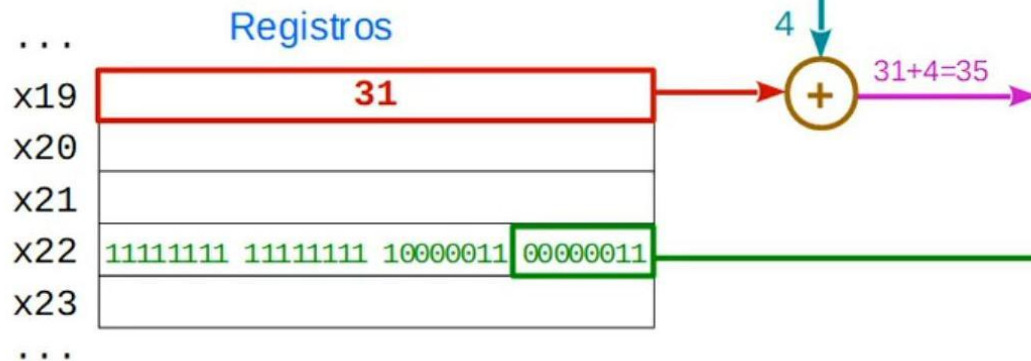
Instrucciones de Almacenamiento

Almacenar byte de registro a memoria

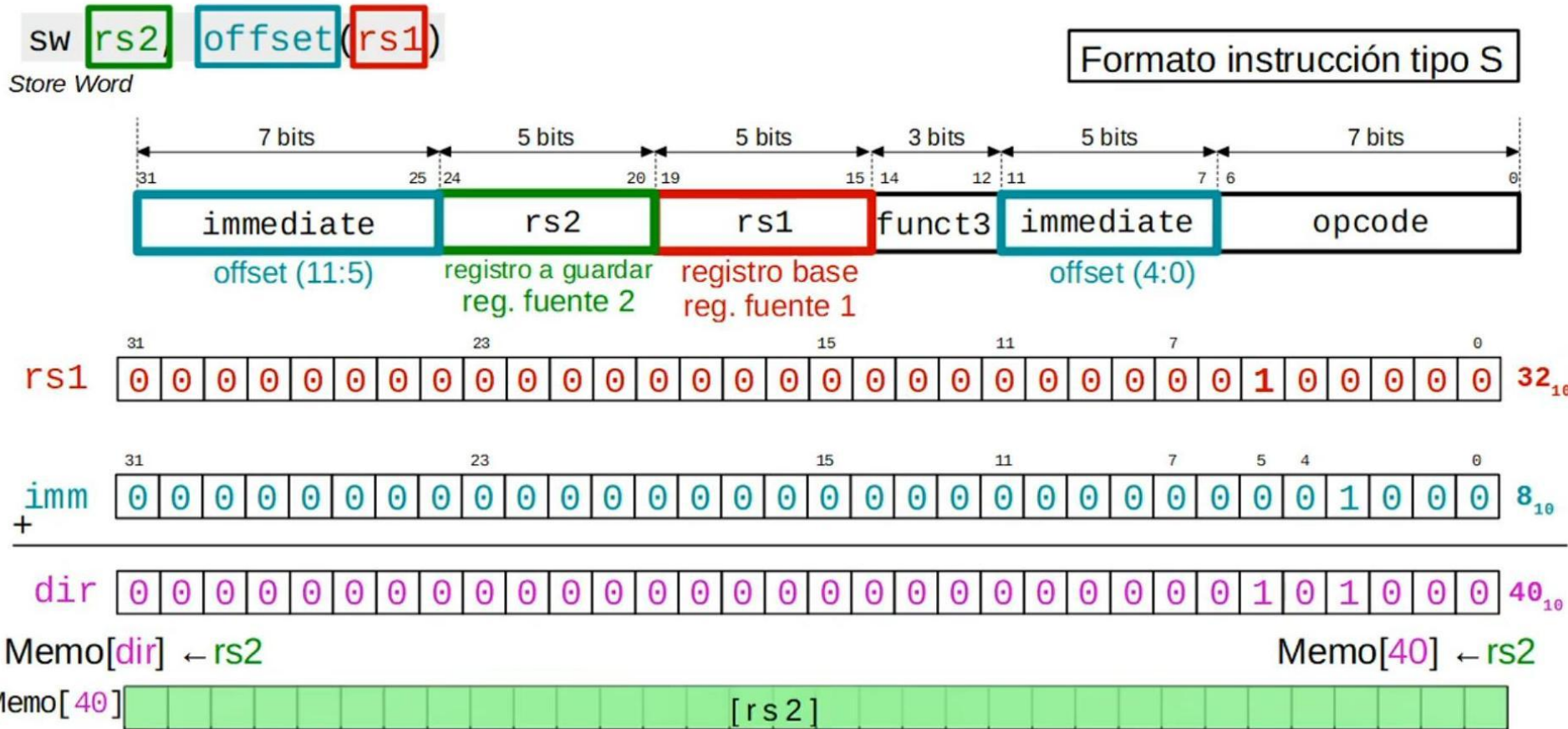
```
sb rs2, offset(rs1)
```

Store Byte

Ejemplo sb x22, 4(x19)



Instrucciones de Almacenamiento



Instrucciones de Carga y Almacenamiento

Tipo I

31		25 24		20 19		15 14		12 11		7 6		0	
imm				rs1		funct3		rd		opcode			
imm(11:0)				rs1		0 0 0		rd		0 0 0 0 0 1 1			
imm(11:0)				rs1		0 0 1		rd		0 0 0 0 0 1 1			
imm(11:0)				rs1		0 1 0		rd		0 0 0 0 0 1 1			
imm(11:0)				rs1		1 0 0		rd		0 0 0 0 0 1 1			
imm(11:0)				rs1		1 0 1		rd		0 0 0 0 0 1 1			

Tipo S

31		25		24		20		19		15		14		12		11		7		6		0			
imm (11:5)						rs2				rs1				funct3		imm (4:0)				opcode					
imm (11:5)						rs2				rs1				0 0 0		imm (4:0)				0 1 0 0 0 1 1					
imm (11:5)						rs2				rs1				0 0 1		imm (4:0)				0 1 0 0 0 1 1					
imm (11:5)						rs2				rs1				0 1 0		imm (4:0)				0 1 0 0 0 1 1					

Instrucciones de Carga y Almacenamiento

Tipo I: Carga de memoria a registro

lw	<i>Load Word</i>
lhu	<i>Load Halfword, Unsigned</i>
lbu	<i>Load Byte, Unsigned</i>
lh	<i>Load Halfword</i>
lb	<i>Load Byte</i>

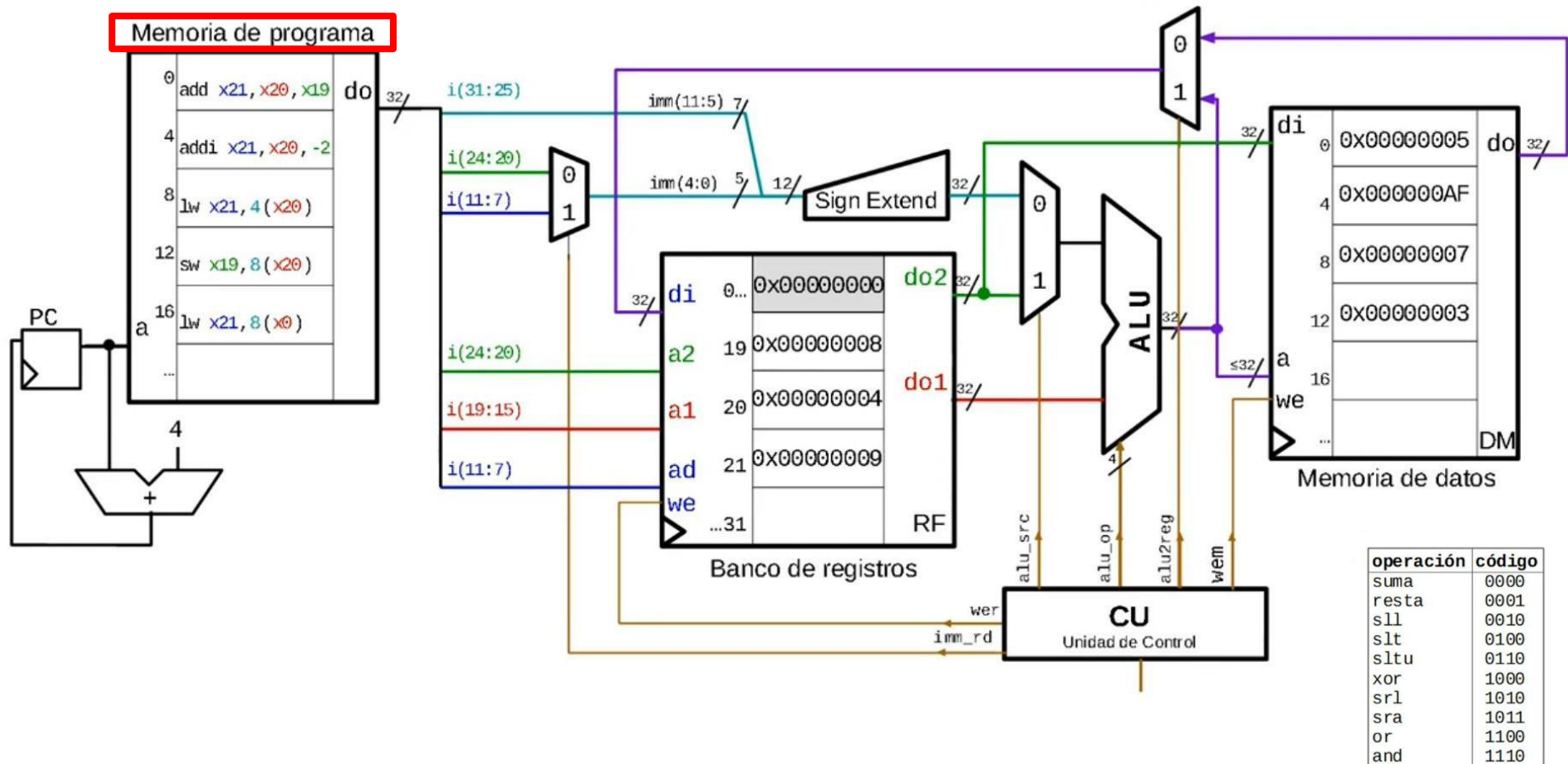
Tipo S: Almacenamiento, de registro a memoria

sw	<i>Store Word</i>
sh	<i>Store Halfword</i>
sb	<i>Store Byte</i>

Instrucciones de Salto

Tipo B		
beq	<i>Branch if Equal</i>	$rs1 == rs2$
bne	<i>Branch if Not Equal</i>	$rs1 \neq rs2$
blt	<i>Branch if Less Than</i>	$rs1 <_s rs2$
bge	<i>Branch if Greater than or Equal</i>	$rs1 \geq_s rs2$
bltu	<i>Branch if Less Than, Unsigned</i>	$rs1 <_u rs2$
bgeu	<i>Branch if Greater than or Equal, Unsigned</i>	$rs1 \geq_u rs2$

Instrucciones de Salto



Instrucciones de Salto

Lenguaje C

```
if (p == q) {  
    f = g + h;  
} else {  
    f = g - h;  
}  
...
```

control de flujo
del programa



Ensamblador

Registro fuente 1 } si son iguales
Registro fuente 2 }
`beq rs1, rs2, LABEL`
Branch if Equal
aquí
salta

Registro fuente 1 } si son distintos
Registro fuente 2 }
`bne rs1, rs2, LABEL`
Branch if Not Equal
aquí
salta

Instrucciones de Salto

Lenguaje C

```
if (p == q) {  
    f = g + h;  
} else {  
    f = g - h;  
}  
...
```

control de flujo
del programa

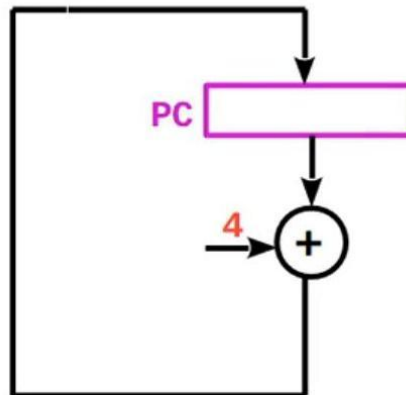
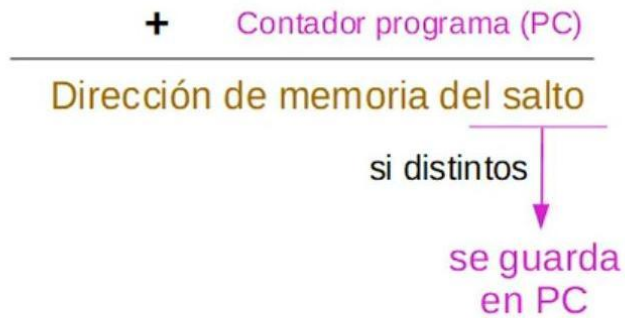
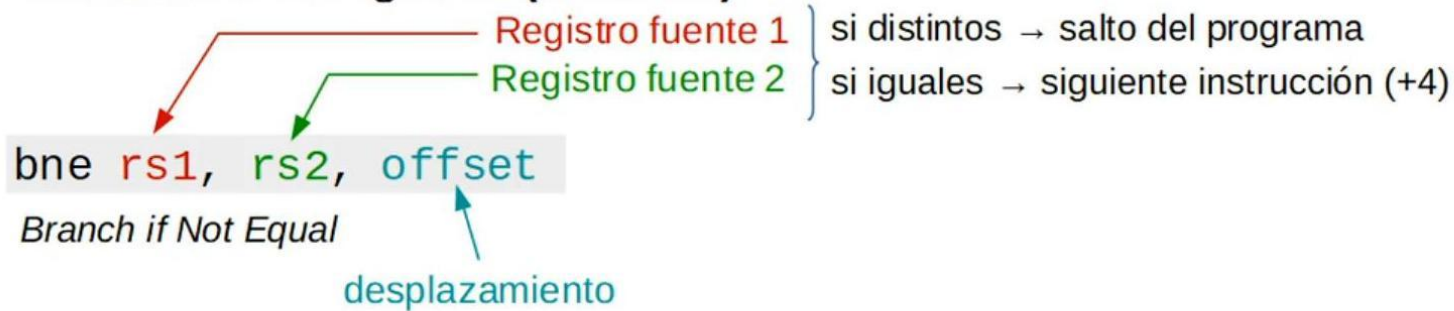


Ensamblador

```
bne x22, x23, Else  
add x19, x20, x21  
beq x0, x0, Exit  
Else:  
    sub x19, x20, x21  
Exit:
```

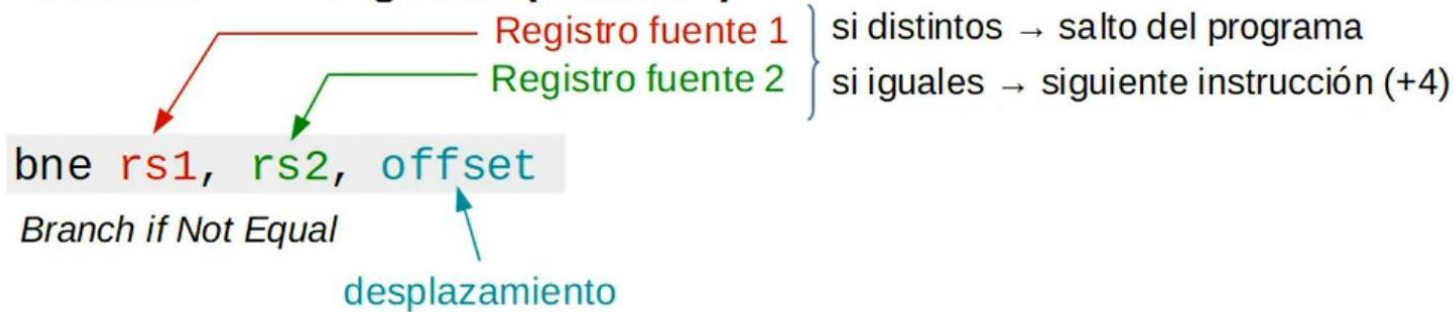
Instrucciones de Salto

Saltar si no son iguales (distintos)

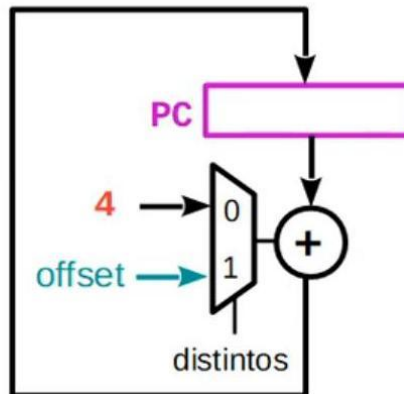


Instrucciones de Salto

Saltar si no son iguales (distintos)



si distintos
se guarda
en PC



Instrucciones de Salto

Saltar si no son iguales (distintos)

Registro fuente 1 } si distintos → salto del programa
Registro fuente 2 } si iguales → siguiente instrucción (+4)

bne rs1, rs2, offset

Branch if Not Equal

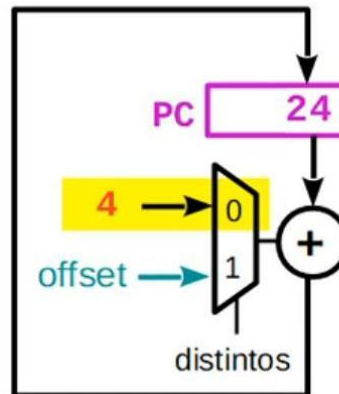
desplazamiento

+ Contador programa (PC)

Dirección de memoria del salto

si distintos

se guarda
en PC



Memoria

20	bne x22, x23, Else
24	add x19, x20, x21
28	beq x0, x0, Exit
32	sub x19, x20, x21
36	
40	
...	

Instrucciones de Salto

Saltar si no son iguales (distintos)

Registro fuente 1 } si distintos → salto del programa
Registro fuente 2 } si iguales → siguiente instrucción (+4)

bne rs1, rs2, offset

Branch if Not Equal

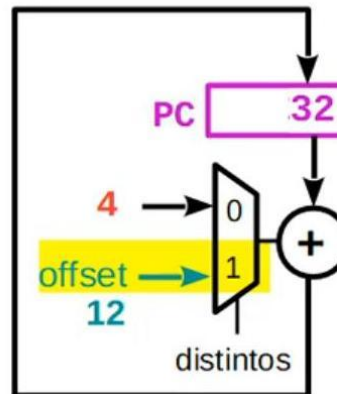
desplazamiento

+ Contador programa (PC)

Dirección de memoria del salto

si distintos

se guarda
en PC



Memoria

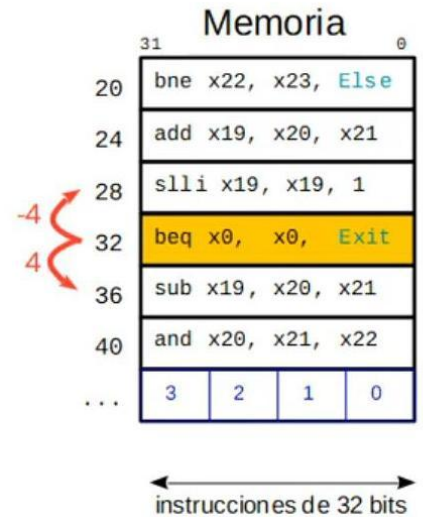
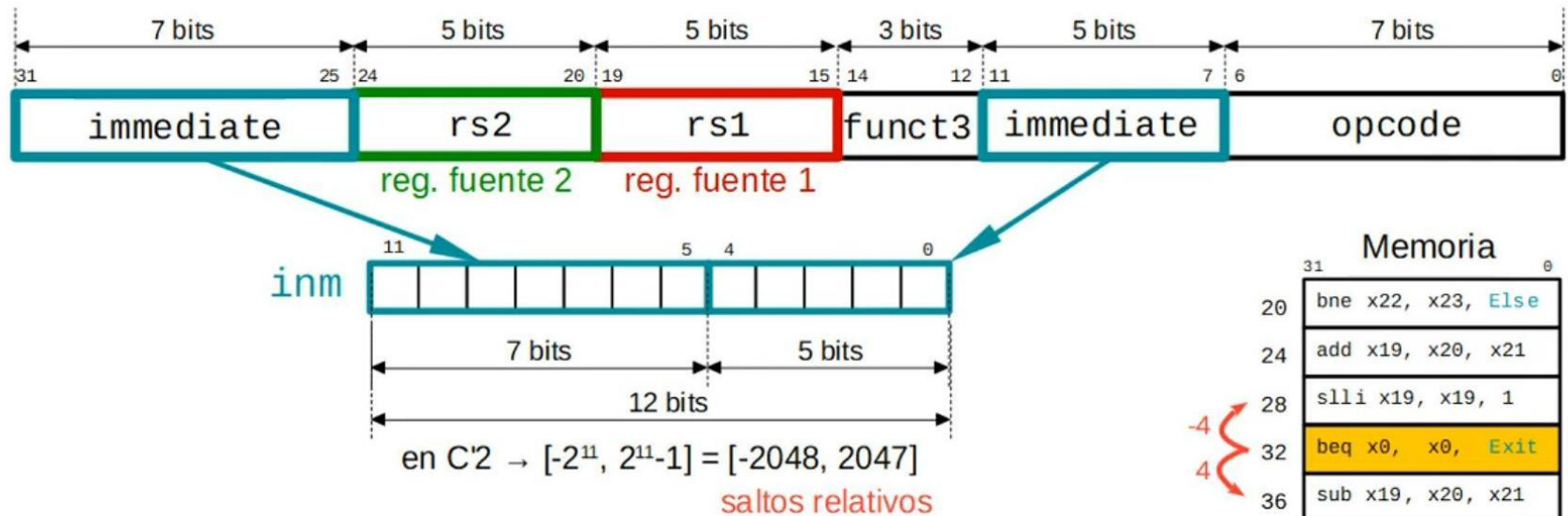
3 instrucciones
offset=12

20	bne x22, x23, Else
24	add x19, x20, x21
28	beq x0, x0, Exit
32	sub x19, x20, x21
36	
40	
...	

Instrucciones de Salto

beq rs1 rs2, offset
Branch if Equal

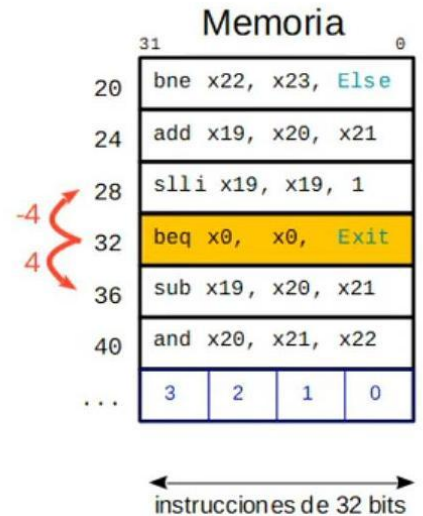
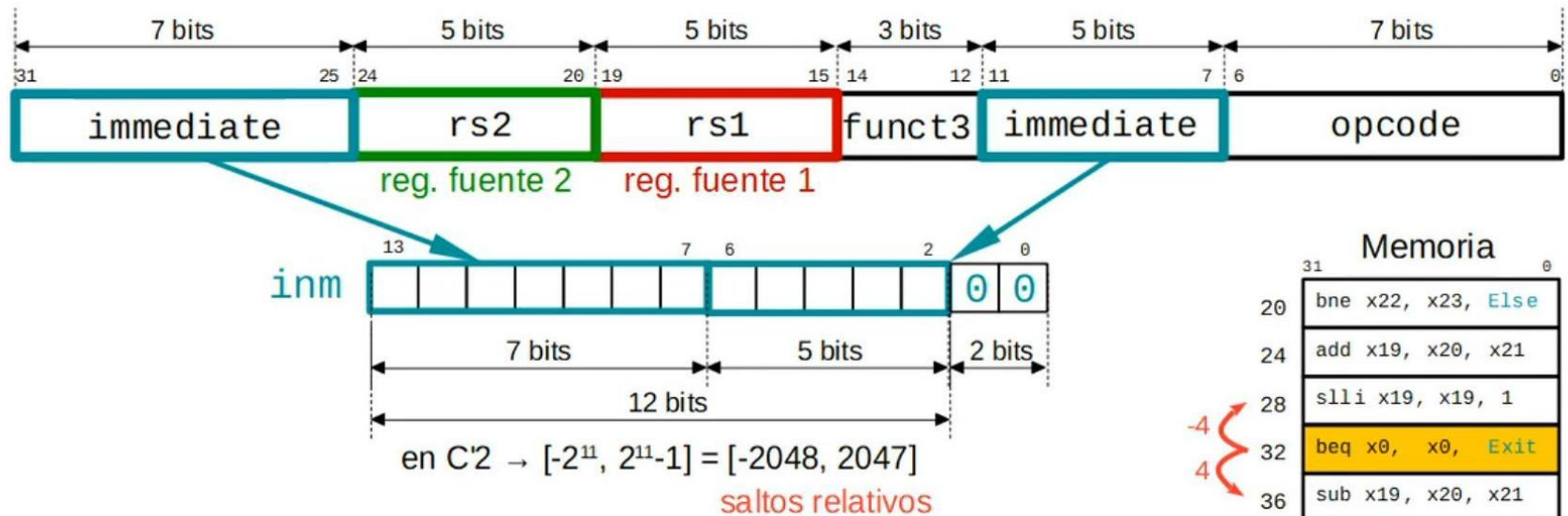
Formato instrucción tipo B



Instrucciones de Salto

beq **rs1** **rs2**, **offset**
Branch if Equal

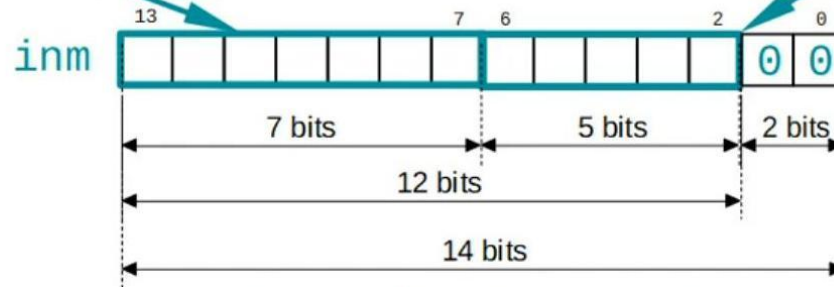
Formato instrucción tipo B



Instrucciones de Salto

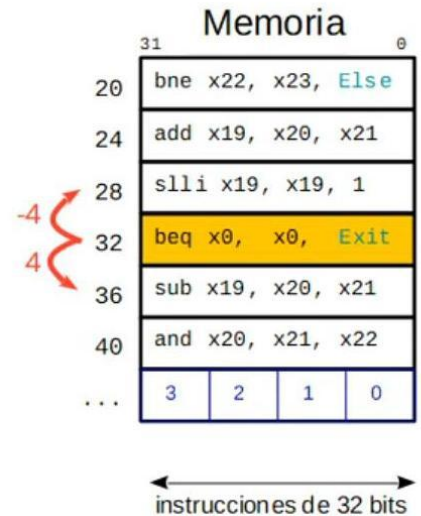
beq rs1 rs2, offset
Branch if Equal

Formato instrucción tipo B



en C'2 $\rightarrow [-2^{13}, 2^{13}-1] = [-8192, 8191]$

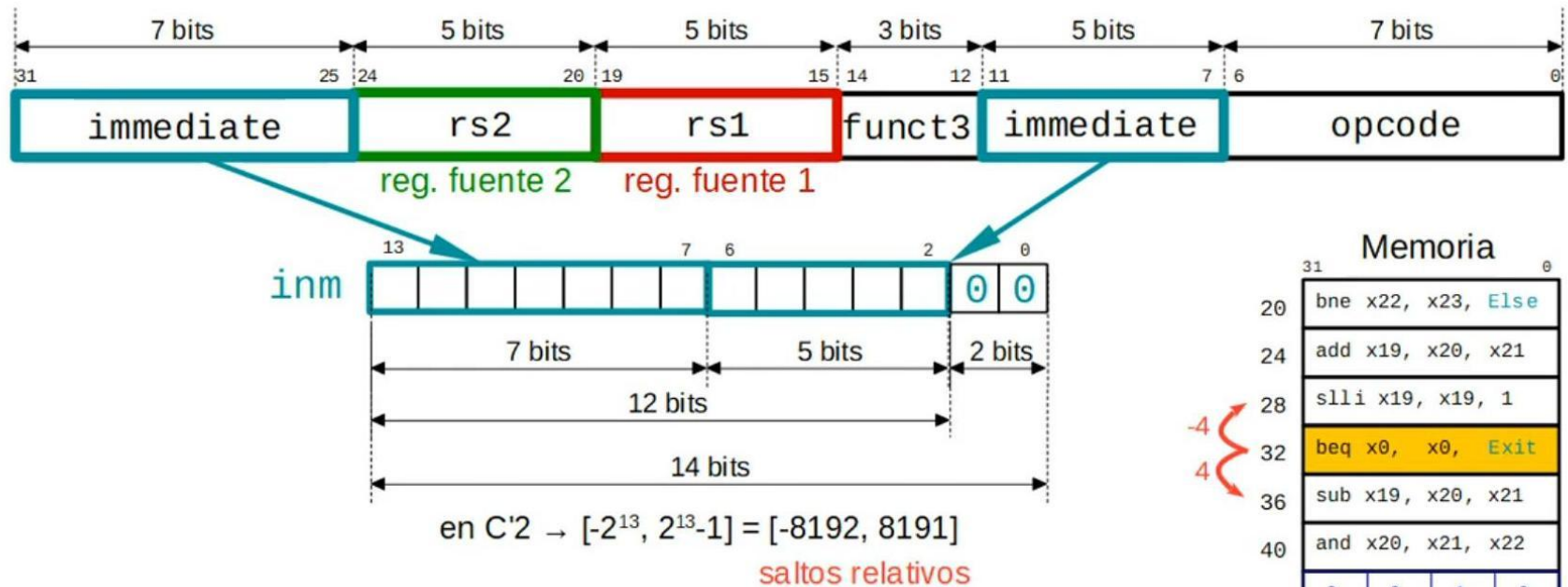
saltos relativos



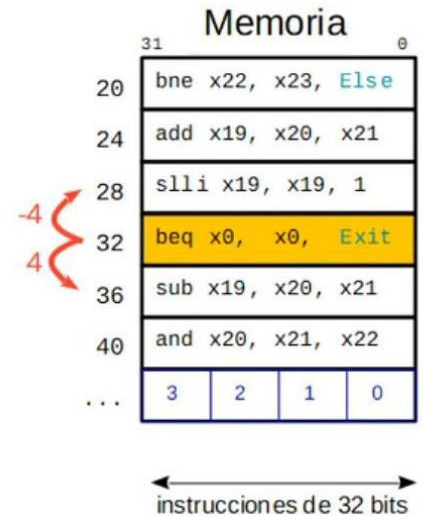
Instrucciones de Salto

beq rs1, rs2, offset
Branch if Equal

Formato instrucción tipo B



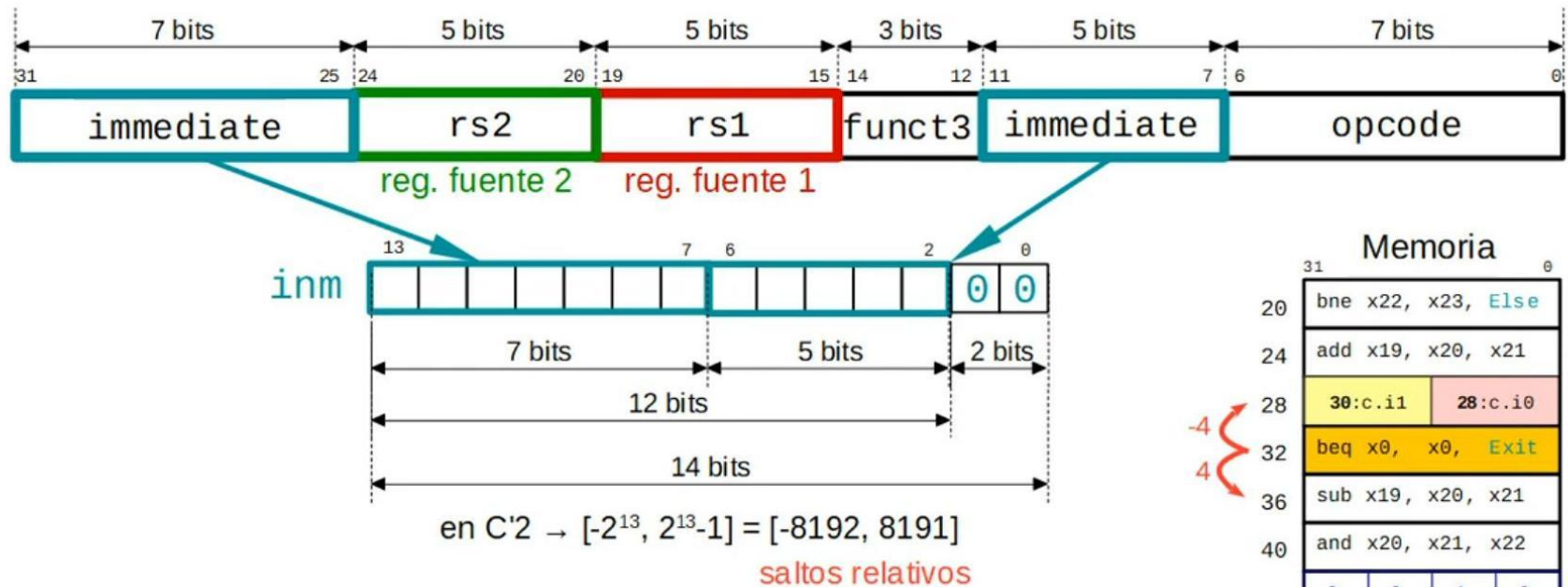
RISC-V tiene una extensión de instrucciones comprimidas de 16 bits: 2 bytes



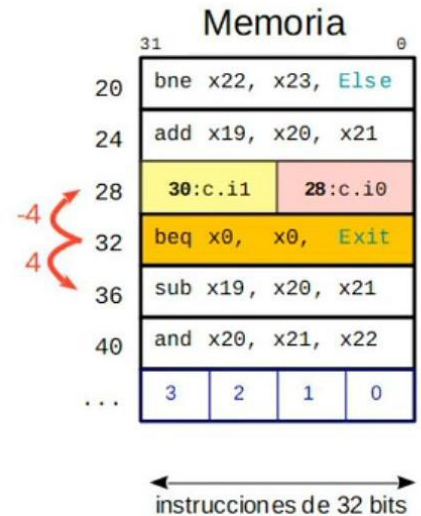
Instrucciones de Salto

beq rs1 rs2, offset
Branch if Equal

Formato instrucción tipo B



RISC-V tiene una extensión de instrucciones comprimidas de 16 bits: 2 bytes

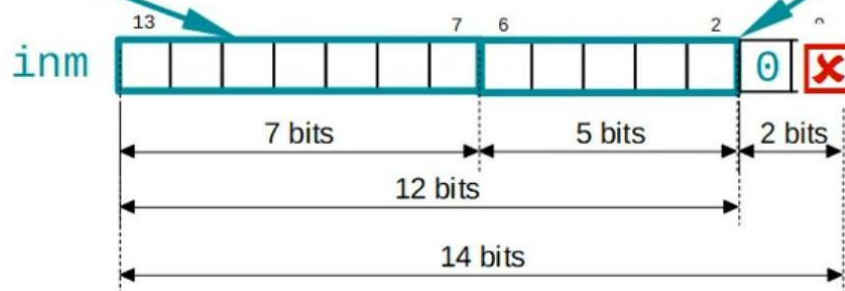


Instrucciones de Salto

beq rs1, rs2, offset

Branch if Equal

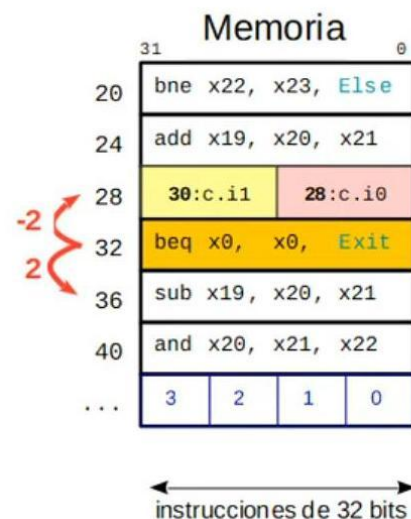
Formato instrucción tipo B



en C'2 $\rightarrow [-2^{13}, 2^{13}-1] = [-8192, 8191]$

saltos relativos

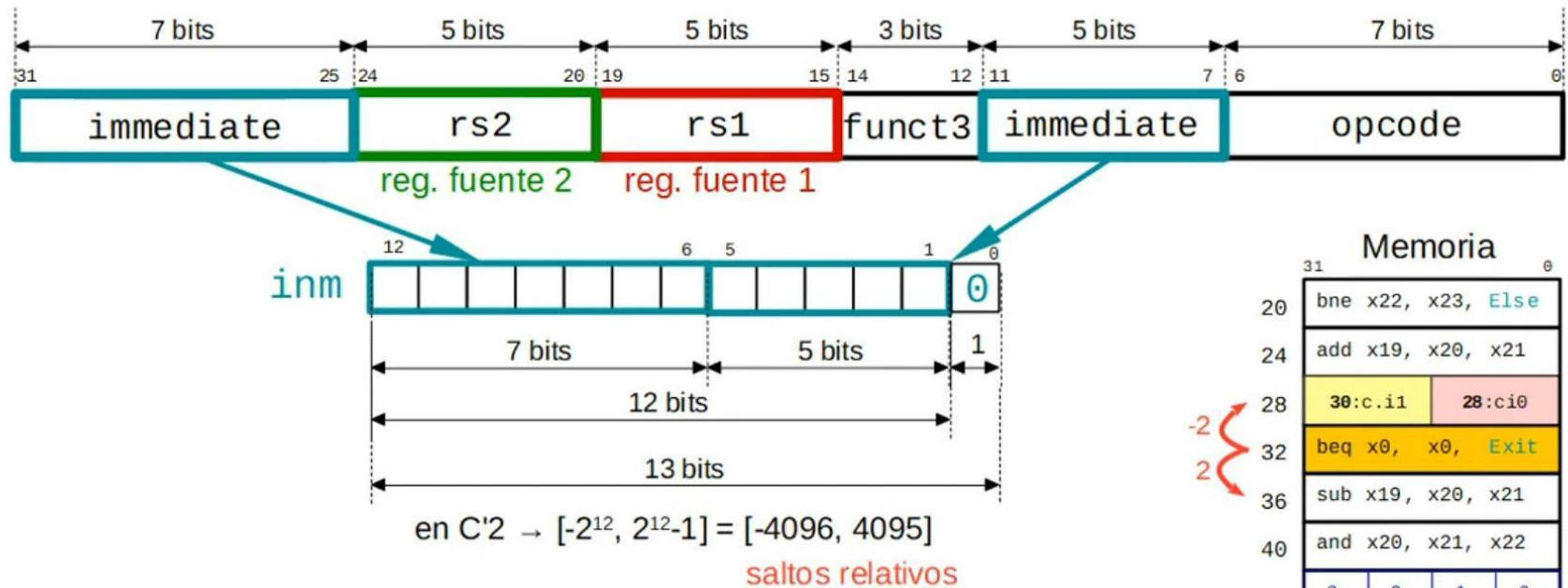
RISC-V tiene una extensión de instrucciones comprimidas de 16 bits: 2 bytes



Instrucciones de Salto

beq rs1, rs2, offset
Branch if Equal

Formato instrucción tipo B



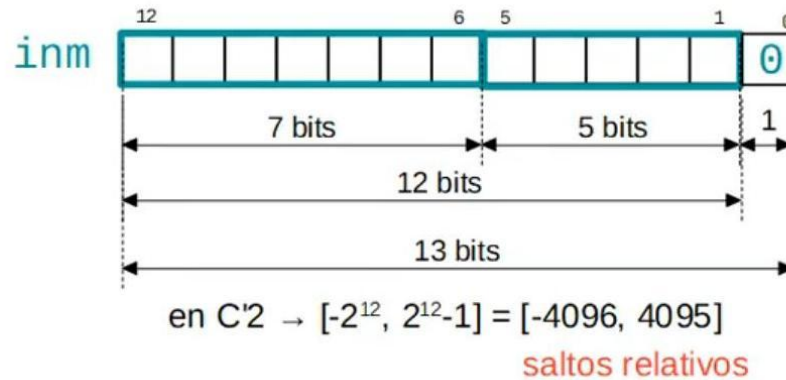
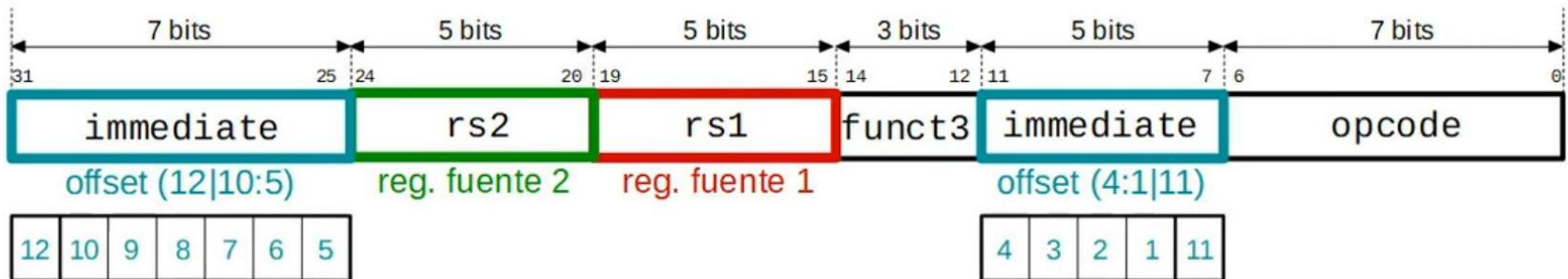
RISC-V tiene una extensión de instrucciones comprimidas de 16 bits: 2 bytes

instrucciones de 32 bits

Instrucciones de Salto

beq rs1, rs2, offset
Branch if Equal

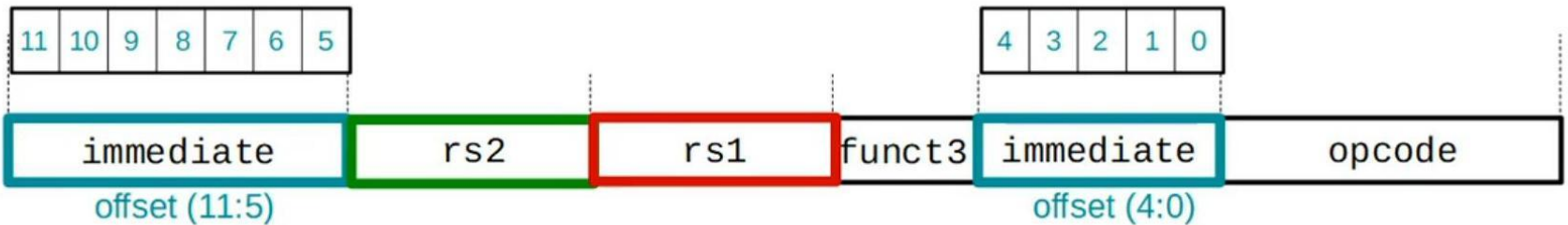
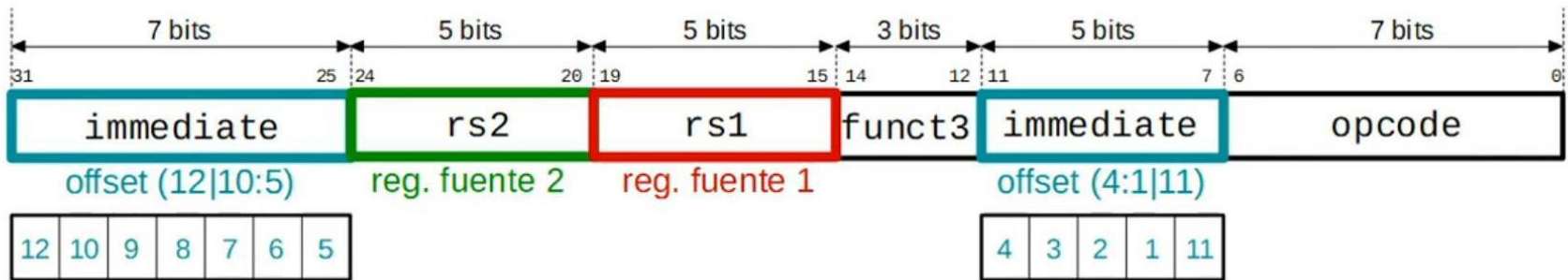
Formato instrucción tipo B



Instrucciones de Salto

beq **rs1**, **rs2**, **offset**
Branch if Equal

Formato instrucción tipo B



sw **rs2**, **offset**(**rs1**)
Store Word

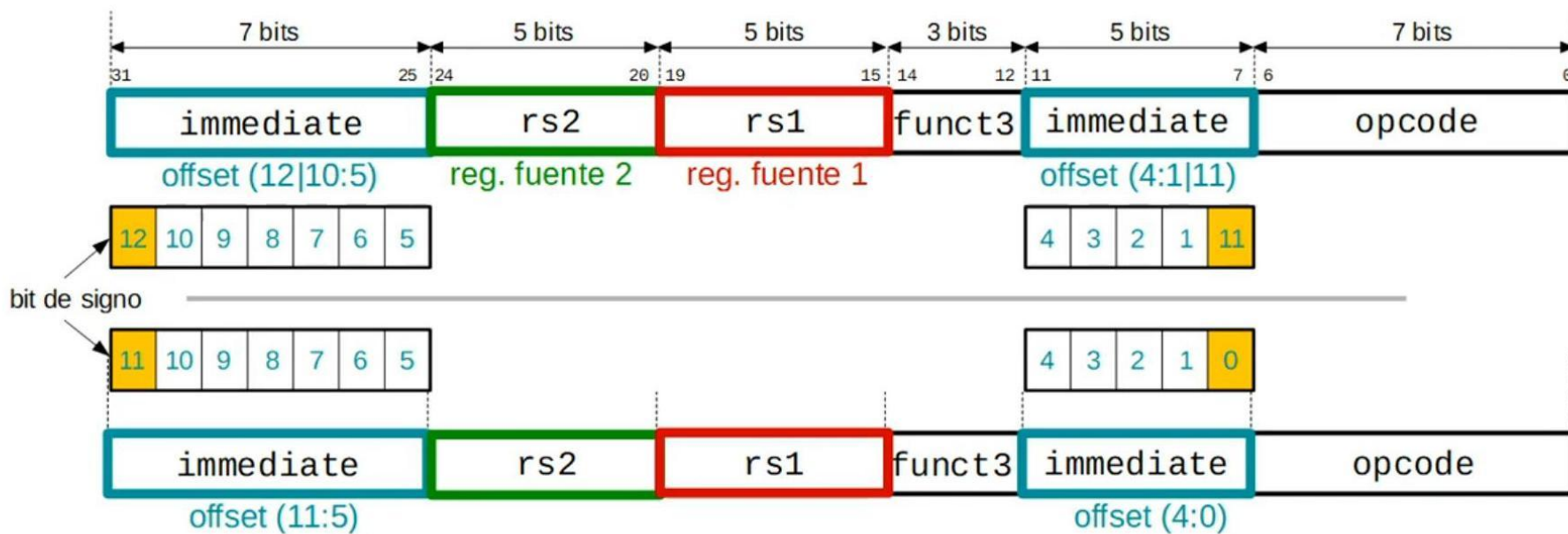
Formato instrucción tipo S

Instrucciones de Salto

beq **rs1**, **rs2**, **offset**

Branch if Equal

Formato instrucción tipo B



sw **rs2**, **offset(rs1)**

Store Word

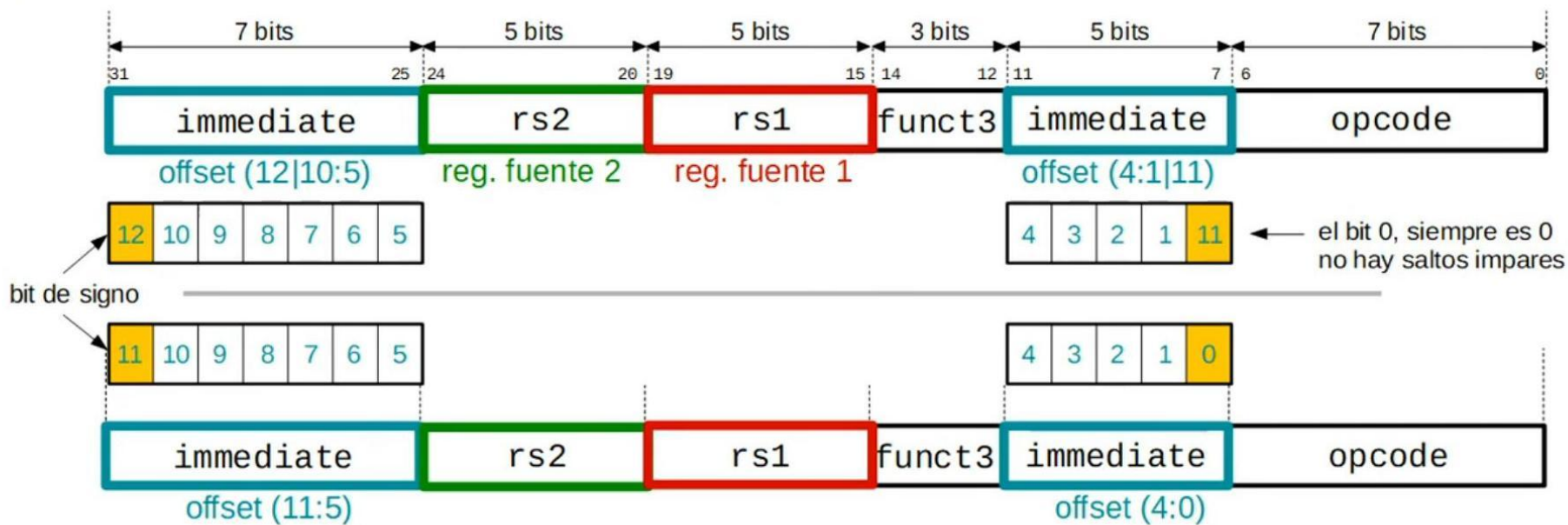
Formato instrucción tipo S

Instrucciones de Salto

beq **rs1** **rs2**, **offset**

Branch if Equal

Formato instrucción tipo B

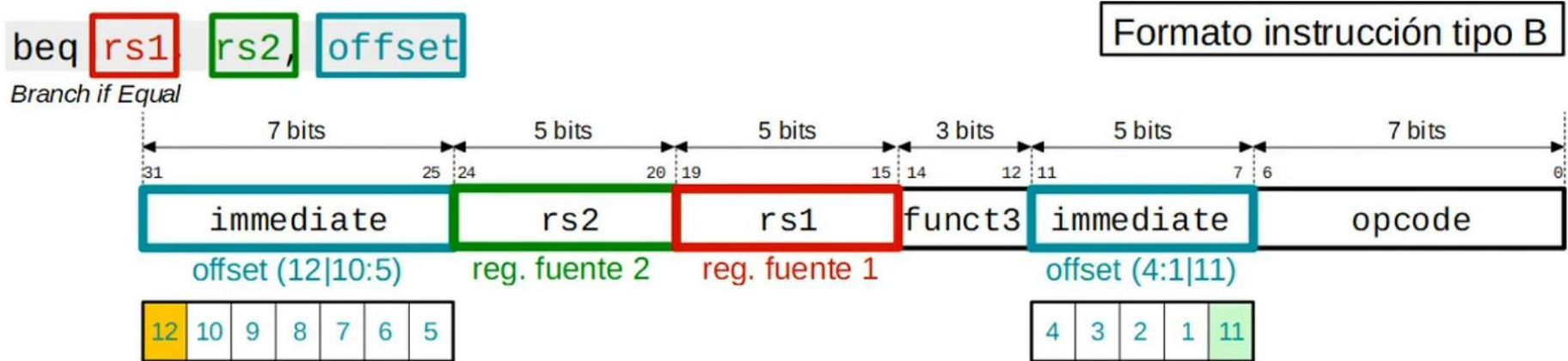


sw **rs2**, **offset**(**rs1**)

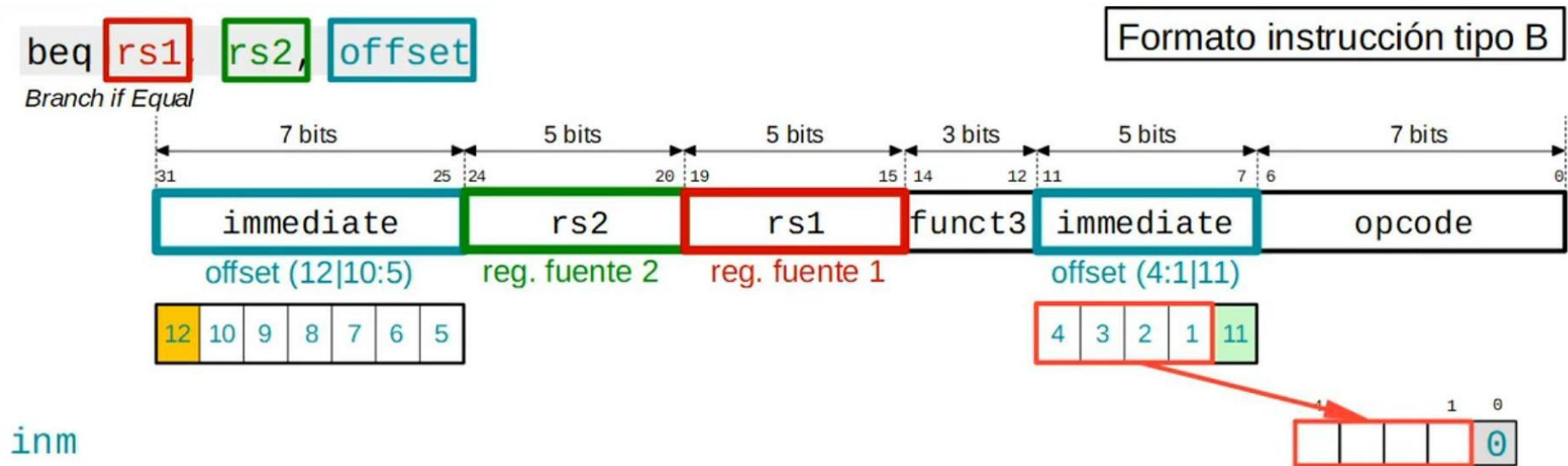
Store Word

Formato instrucción tipo S

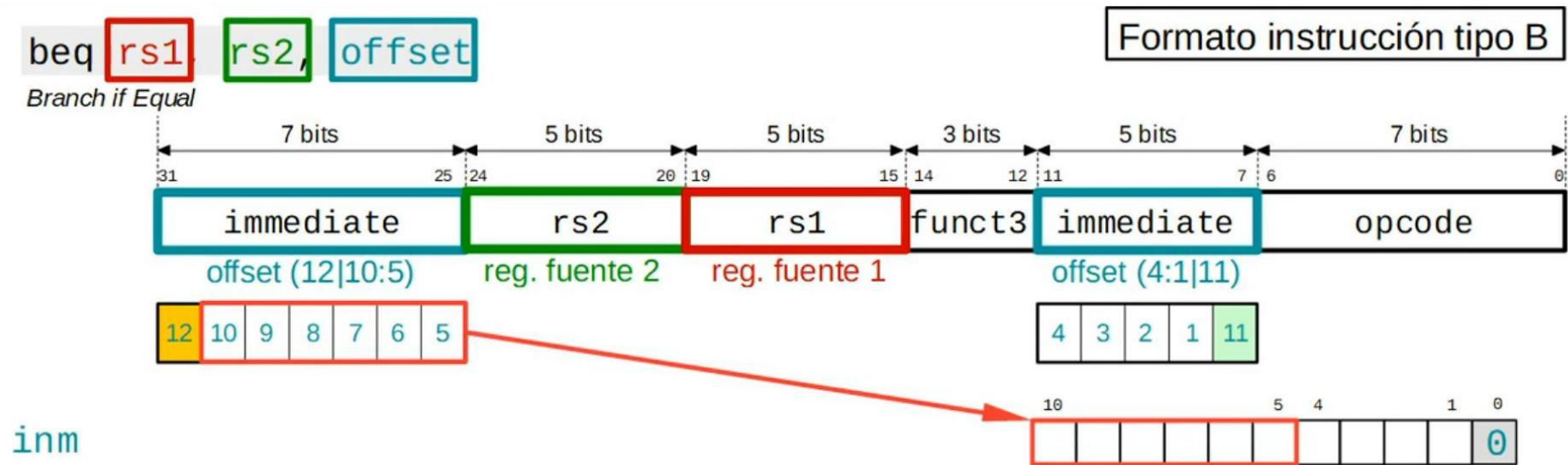
Instrucciones de Salto - Reconstrucción del Inmediato



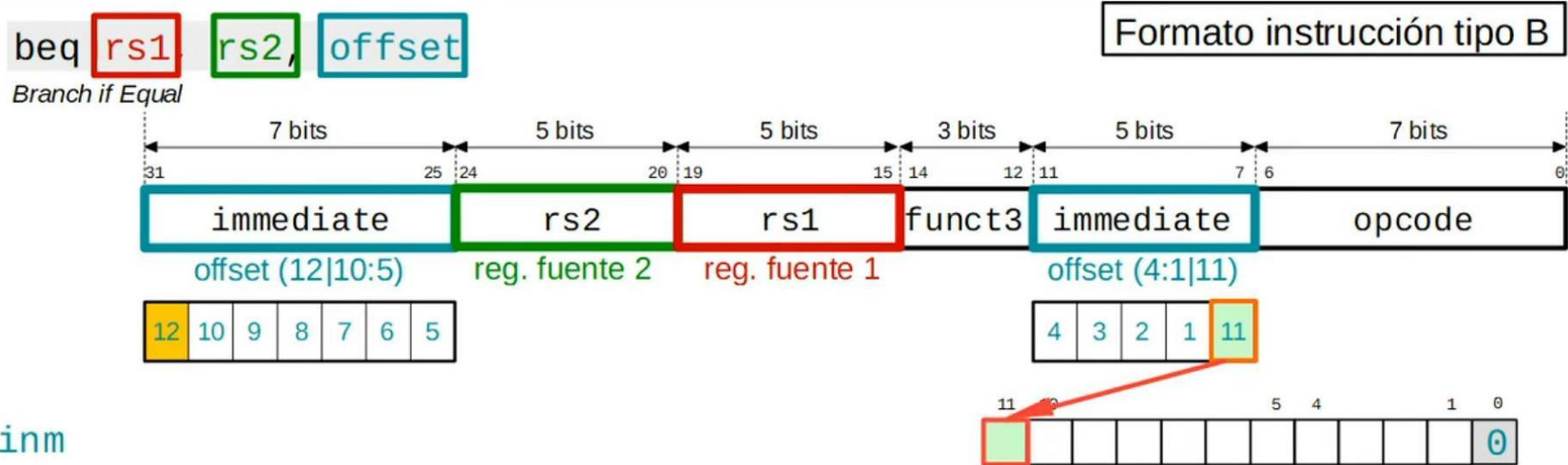
Instrucciones de Salto - Reconstrucción del Inmediato



Instrucciones de Salto - Reconstrucción del Inmediato

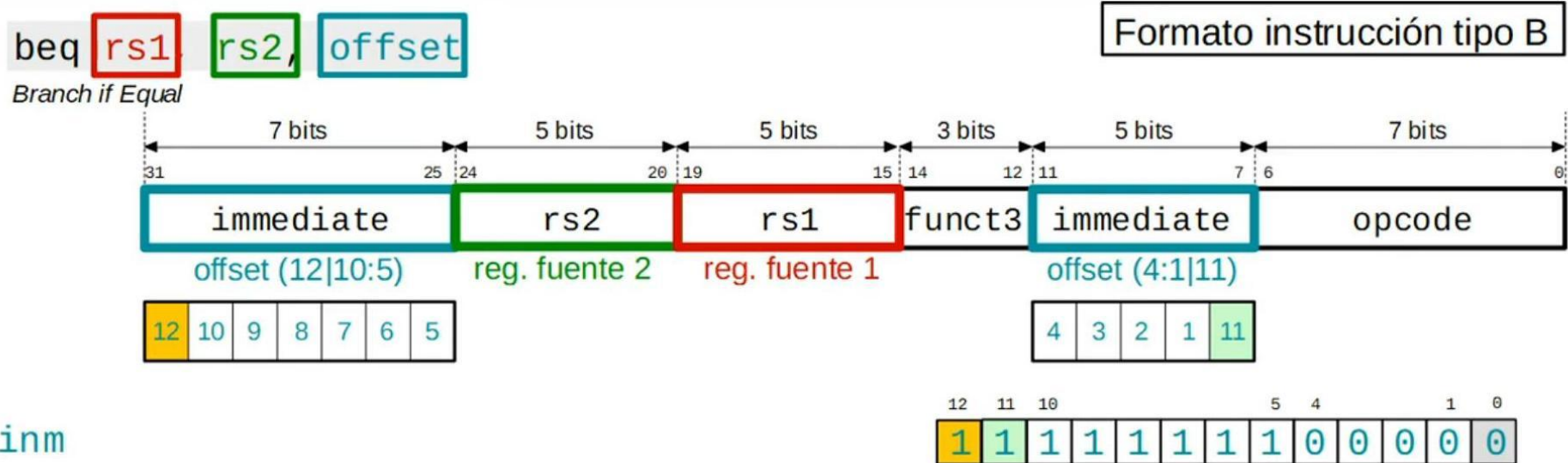


Instrucciones de Salto - Reconstrucción del Inmediato



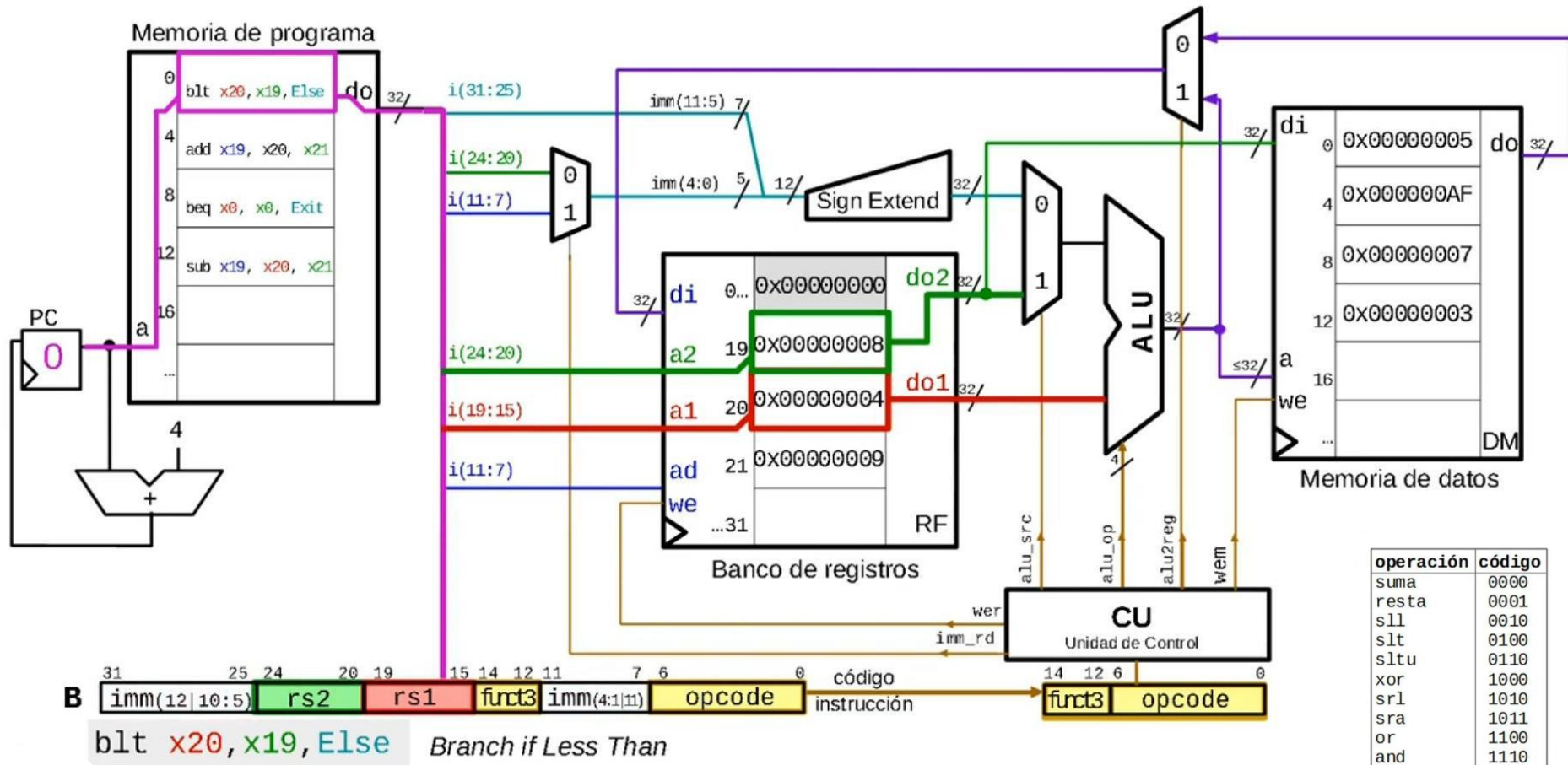


Instrucciones de Salto - Reconstrucción del Inmediato





Instrucciones de Salto



La salida de la ALU debe activar el Mux del PC para sumar el offset

Instrucciones de Salto

Tipo B

31	25	24	20	19	15	14	12	11	7	6	0
imm (12 10:5)		rs2		rs1		funct3		imm (4:1 11)		opcode	
imm (12 10:5)		rs2		rs1		0 0 0		imm (4:1 11)		1 1 0 0 0 1 1	
imm (12 10:5)		rs2		rs1		0 0 1		imm (4:1 11)		1 1 0 0 0 1 1	
imm (12 10:5)		rs2		rs1		1 0 0		imm (4:1 11)		1 1 0 0 0 1 1	
imm (12 10:5)		rs2		rs1		1 0 1		imm (4:1 11)		1 1 0 0 0 1 1	
imm (12 10:5)		rs2		rs1		1 1 0		imm (4:1 11)		1 1 0 0 0 1 1	
imm (12 10:5)		rs2		rs1		1 1 1		imm (4:1 11)		1 1 0 0 0 1 1	

Instrucciones de Salto

Tipo B		
beq	<i>Branch if Equal</i>	$rs1 == rs2$
bne	<i>Branch if Not Equal</i>	$rs1 \neq rs2$
blt	<i>Branch if Less Than</i>	$rs1 <_s rs2$
bge	<i>Branch if Greater than or Equal</i>	$rs1 \geq_s rs2$
bltu	<i>Branch if Less Than, Unsigned</i>	$rs1 <_u rs2$
bgeu	<i>Branch if Greater than or Equal, Unsigned</i>	$rs1 \geq_u rs2$

Lenguaje C

`i = 65000;` ➡ `65000 > 2047`



immediate	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------

12 bits

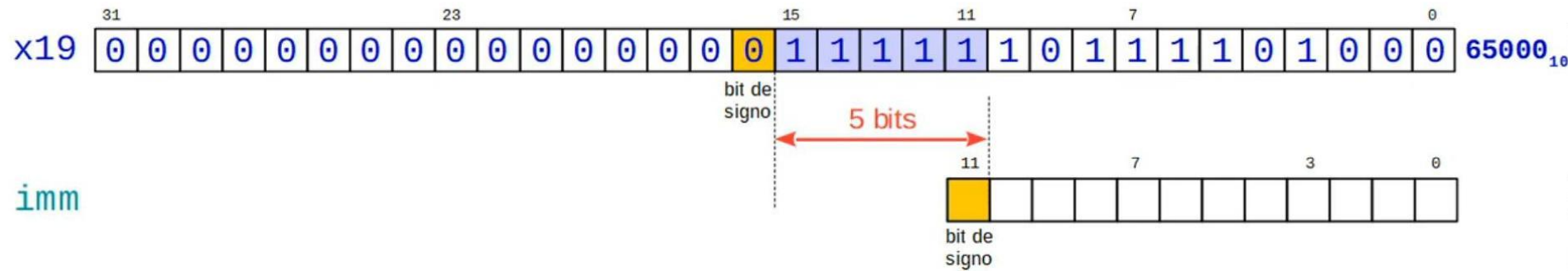
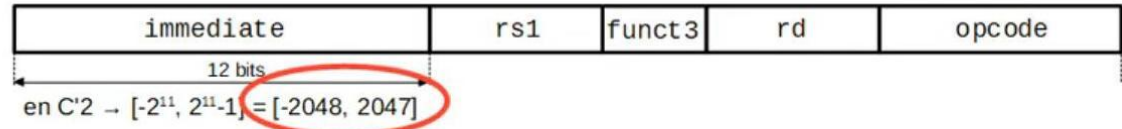
en C'2 $\rightarrow [-2^{11}, 2^{11}-1] = [-2048, 2047]$



Valor Máximo de una Constante

Lenguaje C

`i = 65000;` → $65000 > 2047$

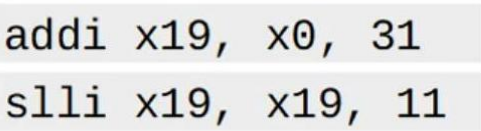
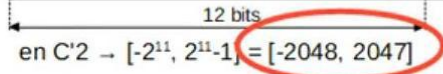


`addi x19, x0, 31`
`slli x19, x19, 11`

→ $31 \cdot 2^{11} = 31 \cdot 2048 = 63488$

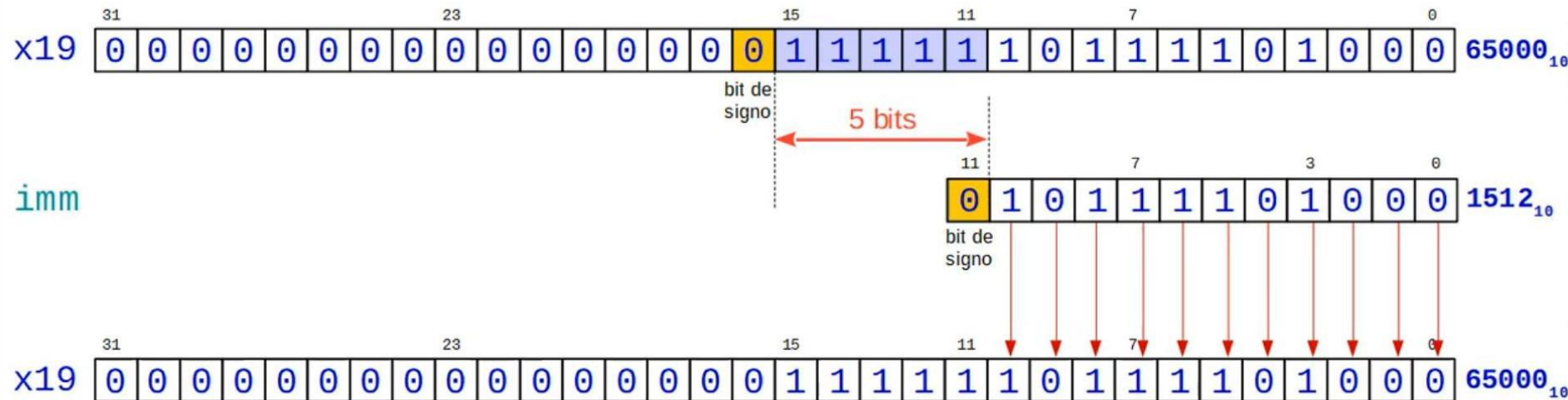
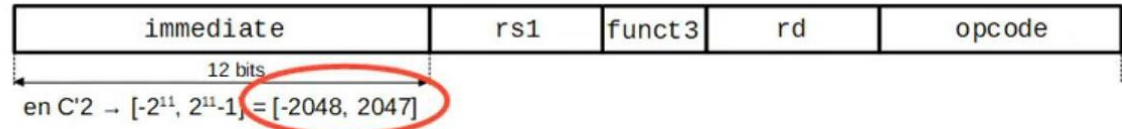
Lenguaje C

`i = 65000;` ➡ `65000 > 2047`



Lenguaje C

`i = 65000;` $\Rightarrow 65000 > 2047$



```
addi x19, x0, 31  
slli x19, x19, 11
```

→ $31 \cdot 2^{11} = 31 \cdot 2048 = 63488$

```
addi x19, x19, 1512
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;  
  
for (int i=0; i<20; i++)  
    sum += A[i];
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

```
BGE    x11, x12, exit
```

```
exit:
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

```
BGE    x11, x12, exit
```

```
exit:
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

```
BGE    x11, x12, exit  
SLL    x13, x11, 2     # i * 4  
ADD    x13, x13, x8    # & of A + i  
LW     x13, 0(x13)     # *(A + i)
```

```
exit:
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

```
BGE    x11, x12, exit  
SLL    x13, x11, 2    # i * 4  
ADD    x13, x13, x8    # & of A + i  
LW     x13, 0(x13)    # *(A + i)  
ADD    x10, x10, x13  # increment sum
```

```
exit:
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

```
BGE    x11, x12, exit  
SLL    x13, x11, 2    # i * 4  
ADD    x13, x13, x8    # & of A + i  
LW     x13, 0(x13)    # *(A + i)  
ADD    x10, x10, x13   # increment sum  
ADDI   x11, x11, 1    # i++
```

```
exit:
```

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

Loop:

```
BGE    x11, x12, exit  
SLL    x13, x11, 2     # i * 4  
ADD    x13, x13, x8    # & of A + i  
LW     x13, 0(x13)     # *(A + i)  
ADD    x10, x10, x13   # increment sum  
ADDI   x11, x11, 1     # i++  
BEQ    x0, x0, loop    # Iterate
```

exit:

Ejemplo de Compilación

```
int A[20];  
int sum = 0;
```

```
for (int i=0; i<20; i++)  
    sum += A[i];
```

```
# Assume x8 holds pointer to A  
# Assign x10=sum, x11=i
```

```
ADD    x10, x0, x0    # sum=0  
ADD    x11, x0, x0    # i=0  
ADDI   x12, x0, 20    # x12=20
```

```
Loop:
```

```
BGE    x11, x12, exit  
SLL    x13, x11, 2    # i * 4  
ADD    x13, x13, x8    # & of A + i  
LW     x13, 0(x13)    # *(A + i)  
ADD    x10, x10, x13   # increment sum  
ADDI   x11, x11, 1    # i++  
BEQ    x0, x0, loop    # Iterate
```

```
exit:
```

Operaciones Aritméticas

Instruction	Type	Example	Meaning
Add	R	add rd, rs1, rs2	$R[rd] = R[rs1] + R[rs2]$
Subtract	R	sub rd, rs1, rs2	$R[rd] = R[rs1] - R[rs2]$
Add immediate	I	addi rd, rs1, imm12	$R[rd] = R[rs1] + \text{SignExt}(\text{imm12})$
Set less than	R	slt rd, rs1, rs2	$R[rd] = (R[rs1] < R[rs2])? 1 : 0$
Set less than immediate	I	slti rd, rs1, imm12	$R[rd] = (R[rs1] < \text{SignExt}(\text{imm12}))? 1 : 0$
Set less than unsigned	R	sltu rd, rs1, rs2	$R[rd] = (R[rs1] <_u R[rs2])? 1 : 0$
Set less than immed unsigned	I	sltiu rd, rs1, imm12	$R[rd] = (R[rs1] <_u \text{SignExt}(\text{imm12}))? 1 : 0$
Load upper immediate	U	lui rd, imm20	$R[rd] = \text{SignExt}(\text{imm20} << 12)$
Add upper immediate to PC	U	auipc rd, imm20	$R[rd] = \text{PC} + \text{SignExt}(\text{imm20} << 12)$

Operaciones Lógicas

Instruction	Type	Example	Meaning
AND	R	and rd, rs1, rs2	$R[rd] = R[rs1] \& R[rs2]$
OR	R	or rd, rs1, rs2	$R[rd] = R[rs1] R[rs2]$
XOR	R	xor rd, rs1, rs2	$R[rd] = R[rs1] \wedge R[rs2]$
AND immediate	I	andi rd, rs1, imm12	$R[rd] = R[rs1] \& \text{SignExt}(\text{imm12})$
OR immediate	I	ori rd, rs1, imm12	$R[rd] = R[rs1] \text{SignExt}(\text{imm12})$
XOR immediate	I	xori rd, rs1, imm12	$R[rd] = R[rs1] \wedge \text{SignExt}(\text{imm12})$
Shift left logical	R	sll rd, rs1, rs2	$R[rd] = R[rs1] \ll R[rs2]$
Shift right logical	R	srl rd, rs1, rs2	$R[rd] = R[rs1] \gg R[rs2]$ (logical)
Shift right arithmetic	R	sra rd, rs1, rs2	$R[rd] = R[rs1] \gg R[rs2]$ (arithmetic)
Shift left logical immediate	I	slli rd, rs1, shamt	$R[rd] = R[rs1] \ll \text{shamt}$
Shift right logical imm.	I	srli rd, rs1, shamt	$R[rd] = R[rs1] \gg \text{shamt}$ (logical)
Shift right arithmetic immed.	I	srai rd, rs1, shamt	$R[rd] = R[rs1] \gg \text{shamt}$ (arithmetic)

Operaciones de Memoria a Registro

Instruction	Type	Example	Meaning
Load doubleword	I	ld rd, imm12(rs1)	$R[rd] = \text{Mem8}[R[rs1] + \text{SignExt}(\text{imm12})]$
Load word	I	lw rd, imm12(rs1)	$R[rd] = \text{SignExt}(\text{Mem4}[R[rs1] + \text{SignExt}(\text{imm12})])$
Load halfword	I	lh rd, imm12(rs1)	$R[rd] = \text{SignExt}(\text{Mem2}[R[rs1] + \text{SignExt}(\text{imm12})])$
Load byte	I	lb rd, imm12(rs1)	$R[rd] = \text{SignExt}(\text{Mem1}[R[rs1] + \text{SignExt}(\text{imm12})])$
Load word unsigned	I	lwu rd, imm12(rs1)	$R[rd] = \text{ZeroExt}(\text{Mem4}[R[rs1] + \text{SignExt}(\text{imm12})])$
Load halfword unsigned	I	lhu rd, imm12(rs1)	$R[rd] = \text{ZeroExt}(\text{Mem2}[R[rs1] + \text{SignExt}(\text{imm12})])$
Load byte unsigned	I	lbu rd, imm12(rs1)	$R[rd] = \text{ZeroExt}(\text{Mem1}[R[rs1] + \text{SignExt}(\text{imm12})])$

Operaciones de Registro a Memoria

Instruction	Type	Example	Meaning
Store doubleword	S	sd rs2, imm12(rs1)	$\text{Mem8}[\text{R}[\text{rs1}] + \text{SignExt}(\text{imm12})] = \text{R}[\text{rs2}]$
Store word	S	sw rs2, imm12(rs1)	$\text{Mem4}[\text{R}[\text{rs1}] + \text{SignExt}(\text{imm12})] = \text{R}[\text{rs2}](31:0)$
Store halfword	S	sh rs2, imm12(rs1)	$\text{Mem2}[\text{R}[\text{rs1}] + \text{SignExt}(\text{imm12})] = \text{R}[\text{rs2}](15:0)$
Store byte	S	sb rs2, imm12(rs1)	$\text{Mem1}[\text{R}[\text{rs1}] + \text{SignExt}(\text{imm12})] = \text{R}[\text{rs2}](7:0)$

Operaciones de Control de Flujo

Instruction	Type	Example	Meaning
Branch equal	SB	beq rs1, rs2, imm12	if (R[rs1] == R[rs2]) PC = PC + SignExt(imm12 << 1)
Branch not equal	SB	bne rs1, rs2, imm12	if (R[rs1] != R[rs2]) PC = PC + SignExt(imm12 << 1)
Branch greater than or equal	SB	bge rs1, rs2, imm12	if (R[rs1] >= R[rs2]) PC = PC + SignExt(imm12 << 1)
Branch greater than or equal unsigned	SB	bgeu rs1, rs2, imm12	if (R[rs1] >= _u R[rs2]) PC = PC + SignExt (imm12 << 1)
Branch less than	SB	blt rs1, rs2, imm12	if (R[rs1] < R[rs2]) PC = PC + SignExt(imm12 << 1)
Branch less than unsigned	SB	bltu rs1, rs2, imm12	if (R[rs1] < _u R[rs2]) PC = PC + SignExt(imm12 << 1)
Jump and link	UJ	jal rd, imm20	R[rd] = PC + 4 PC = PC + SignExt(imm20 << 1)
Jump and link register	I	jalr rd, imm12(rs1)	R[rd] = PC + 4 PC = (R[rs1] + SignExt(imm12)) & (-1)

Pseudo Operaciones

Pseudo-instruction	Base instruction(s)	Meaning
li rd, imm	addi rd, x0, imm	Load immediate
mv rd, rs	addi rd, rs, 0	Copy register
not rd, rs	xori rd, rs, -1	One's complement
neg rd, rs	sub rd, x0, rs	Two's complement
bgt{u} rs, rt, offset	blt{u} rt, rs, offset	Branch if > (u: unsigned)
ble{u} rs, rt, offset	bge{u} rt, rs, offset	Branch if $\geq$ (u: unsigned)
b{eq ne}z rs, offset	b{eq ne} rs, x0, offset	Branch if { = $\neq$ }
b{ge lt}z rs, offset	b{ge lt} rs, x0, offset	Branch if { $\geq$ < }
b{le gt}z rs, offset	b{ge lt} x0, rs, offset	Branch if { $\leq$ > }
j offset	jal x0, offset	Unconditional jump
call offset	jal ra, offset	Call subroutine (near)
ret	jalr x0, 0(ra)	Return from subroutine
nop	addi x0, x0, 0	No operation

Pseudo Operaciones

Pseudo-instruction	Base instruction(s)	Meaning
li rd, imm	addi rd, x0, imm	Load immediate
mv rd, rs	addi rd, rs, 0	Copy register
not rd, rs	xori rd, rs, -1	One's complement
neg rd, rs	sub rd, x0, rs	Two's complement
bgt{u} rs, rt, offset	blt{u} rt, rs, offset	Branch if > (u: unsigned)
ble{u} rs, rt, offset	bge{u} rt, rs, offset	Branch if ≥ (u: unsigned)
b{eq ne}z rs, offset	b{eq ne} rs, x0, offset	Branch if { = ≠ }
b{ge lt}z rs, offset	b{ge lt} rs, x0, offset	Se implementa con Jump and Link JAL ra, offset 1º guarda Dir de retorno ra=PC+4 2º salta a la subrutina PC=PC+(offset<<1)
b{le gt}z rs, offset	b{le gt} x0, rs, offset	
j offset	jal x0, offset	
call offset	jal ra, offset	Call subroutine (near)
ret	jalr x0, 0(ra)	Return from subroutine
nop	addi x0, x0, 0	No operation

Pseudo Operaciones

Pseudo-instruction	Base instruction(s)	Meaning
li rd, imm	addi rd, x0, imm	Load immediate
mv rd, rs	addi rd, rs, 0	Copy register
not rd, rs	xori rd, rs, -1	One's complement
neg rd, rs	sub rd, x0, rs	Two's complement
bgt{u} rs, rt, offset	blt{u} rt, rs, offset	Branch if > (u: unsigned)
ble{u} rs, rt, offset	bge{u} rt, rs, offset	Branch if ≥ (u: unsigned)
b{eq ne}z rs, offset	b{eq ne} rs, x0, offset	Branch if { = ≠ }
b{ge lt}z rs, offset	b{ge lt} rs, x0, offset	Branch if { ≥ < }
b{le gt}z rs, offset	b{ge lt} x0, rs,	Se implementa con Jump and Link Register JALR x0, 0(ra) 1º “guarda” Dir de retorno x0=PC+4 2º salta a la dirección PC=(ra+offset)
j offset	jal x0, offset	
call offset	jal ra, offset	
ret	jalr x0, 0(ra)	Return from subroutine
nop	addi x0, x0, 0	No operation

Pseudo Operaciones

Pseudo-instruction	Base instruction(s)	Meaning
li rd, imm	addi rd, x0, imm	Load immediate
mv rd, rs	addi rd, rs, 0	Copy register
not rd, rs	xori rd, rs, -1	One's complement
neg rd, rs	sub rd, x0, rs	Two's complement
bgt{u} rs, rt, offset	blt{u} rt, rs, offset	Branch if > (u: unsigned)
ble{u} rs, rt, offset	bge{u} rt, rs, offset	Branch if ≥ (u: unsigned)
b{eq ne}z rs, offset	b{eq ne} rs, x0, offset	Se implementa con Jump and Link JAL x0, offset 1º “<i>guarda</i>” Dir de retorno x0=PC+4 2º salta a la dirección PC=PC+(offset<<1)
b{ge lt}z rs, offset	b{ge lt} rs, x0, offset	
b{le gt}z rs, offset	b{ge lt} x0, rs, offset	
j offset	jal x0, offset	
call offset	jal ra, offset	Call subroutine (near)
ret	jalr x0, 0(ra)	Return from subroutine
nop	addi x0, x0, 0	No operation

Arquitectura RISC

Reduced Instruction Set Computer

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta